



**COMSATS University Islamabad, COMSATS
Road, off GT Road, Sahiwal, Pakistan**

Curify

**A project presented to
COMSATS Institute of Information Technology, Islamabad**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2022-2026)

By

Muhammad Saim (CIIT/SP22-BCS-143/SWL)

Waris Ali (CIIT/SP22-BCS-119/SWL)

Adeel Ahmed (CIIT/SP22-BCS-017/SWL)

Declaration

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Saim

Waris Ali

Adeel Ahmed

Certificate of Approval

It is to certify that the final year project of BS (CS) “CURIFY” was developed by Waris Ali (CIIT/SP22-BCS/C/119) Adeel Ahmed (CIIT/SP22-BCS/C/017) and Muhammad Saim CIIT/SP22-BCS/C/143) under the supervision of “Ms. Maryam Mazhar” and co supervisor “Mr. Ali Sher Kashif” and that in (their/his/her) opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

Executive Summary

In public places, there is often a need for monitoring people and different activities going on, which can be referred later for many reasons including security. Appointing humans for this task involves many problems such as increased employee hiring, accuracy problem, trust, no proof for later use, and also the fact that a human can remember things till a certain time limit. Talking about the current security system, they use dumb still cameras with a continuous recording facility irrespective of the fact that any event may happen or not. Moreover they are usually pointing at a specific user defined locations so more than one cameras are required to cover the entire region.

To prevent all these problems from prevailing, the CSCS is developed. It is a surveillance system, which provides solution to many of these problems. It is a stand-alone application which doesn't require any computer to operate. It monitors different situations using a camera which is able to rotate intelligently based on sensor messages and captures the scene in the form of video or photos later reference as well.

Customizable Surveillance Control System (CSCS) is a surveillance system that can be assigned a sensor type as in our case a heat sensor is used, it works accordingly, rotates the camera upon event detection and perform user defined actions like capturing video and stores them, for the future use.

It is an embedded system consisting of Linux fox kit with embedded a running server application also a camera, USB storage device and a sensor node base station is attached with fox kit. LAN communication is used by user to download the videos and to operate the system manually.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Ms. Maryam Mazhar” and our Co-Supervisor “Mr. Ali Sher Kashif”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Saim

Waris Ali

Adeel Ahmed

Abbreviations

SRS	Software Requirement Specification
PC	Personal Computer
UI	User Interface
UX	User Experience
DB	Database
API	Application Programming Interface
Kotlin	Official JVM-based language for Android development
MVVM	Model–View–ViewModel

Table of Contents

Contents

Chapter 1.....	5
Introduction	5
Introduction	6
1.1 Brief	6
1.2 Relevance to Course Modules.....	6
1.3 Project Background.....	6
1.4 Literature Review.....	6
1.5 Analysis from Literature Review	7
1.6 Methodology and Software Lifecycle for This Project.....	7
Chapter 2.....	8
Problem Definition	8
2 Problem Definition.....	9
2.1 Problem Statement	9
2.2 Deliverables and Development Requirements	9
2.3 Current System.....	9
Chapter 3.....	10
Requirement Analysis	10
3 Requirement Analysis.....	11
3.1 Detailed Use Case	11
3.2 Functional Requirements	16
3.3 Non-Functional Requirements	19
Chapter 4.....	20
Design and Architecture	20
4 Design and Architecture.....	21
4.1 System Architecture	21
4.2 Data Dictionary	23
4.3 Process Flow/Representation	25
4.4 Design Models	26
4.4.1 Class Diagram.....	26
4.4.2 ERD Diagram	27
4.4.3 Sequence Diagram	28
4.4.4 State Transition Diagram	29
4.4.5 Data Flow Diagram.....	30
Chapter 5.....	33

Implementation.....	33
5 Implementation	34
5.1 Algorithms	34
5.2 External APIs	39
5.3 User Interface.....	41
Chapter 6.....	55
Testing and Evaluation	55
6. Testing And Evaluation	56
6.1 Manual Testing	56
6.1.1 System Testing.....	56
6.1.2 Unit Testing	57
6.1.3 Functional Testing	58
6.1.4 Integration Testing.....	59
6.2 Automated Testing.....	60
Chapter 7.....	63
Conclusion And Future Work.....	63
7 Conclusion Aan Future Work	64
7.1 Conclusion	64
7.2 Future Work.....	66
References	70

List of Figure

Figure 1 System Architecture.....	21
Figure 2 MVVM Architecture	22
Figure 3 Use Case Diagram For Curify	23
Figure 4 Activity Diagram	25
Figure 5 Class Diagram	26
Figure 6 ERD Diagram	27
Figure 7 Sequence Diagram.....	28
Figure 8 State Transition Diagram.....	29
Figure 9 DFD Level 0.....	30
Figure 10 DFD Level 1.....	31
Figure 11 DFD Level 2.....	32
Figure 12 Medicine Shop.....	41
Figure 13 Admin Panel	42
Figure 14 User Profile.....	43
Figure 15 Payment Method.....	43
Figure 16 CureAI	44
Figure 17 Pending Orders	45
Figure 18 Splash Screen.....	45
Figure 19 Manage Appointment	46
Figure 20 Add Medicine	46
Figure 21 Confirmed Medicine Order.....	47
Figure 22 Doctor Details.....	48
Figure 23 Doctors	48
Figure 24 Login Screen.....	49
Figure 25 Payment Proof	49
Figure 26 Payment Status	50
Figure 27 Payment Details.....	50
Figure 28 Payment Api Server.....	51
Figure 29 Payment Page	52
Figure 30 Payment Confirmed.....	52
Figure 31 Order Details	53
Figure 32 Delivery Options.....	54

List of Tables

Table 1 Login.....	11
Table 2 Search for Doctors	11
Table 3 Book an Appointment	12
Table 4 Order Medicines.....	12
Table 5 Medicine Details using AI	13
Table 6 Manage Appointments	14
Table 7 Process Medicine Orders	14
Table 8 Cancel appointments	14
Table 9 Appointment Booking Payment.....	15
Table 10 Delivery Tracking (Future Module).....	16
Table 11 Customer’s Module Functionalities	16
Table 12 Doctor’s Module Functionalities	17
Table 13 Medical Store Module Functionalities	17
Table 14 Admin Module Functionalities.....	17
Table 15 Delivery Personnel Module Functionalities	18
Table 16 AI Module.....	18
Table 17 Non Functional Requirement	19
Table 18 Data Dictionary	24
Table 19 User Authentication Algorithm	34
Table 20 Appointment Booking Algorithm	35
Table 21 Digital Prescription Upload Algorithm	35
Table 22 Medicine Ordering Algorithm.....	36
Table 23 AI-Based Disease Prediction Algorithm	36
Table 24 Doctor Verification & Approval Algorithm.....	37
Table 25 Notification Handling Algorithm (FCM)	37
Table 26 Order Tracking Algorithm	38
Table 27 Chat / Support Algorithm.....	38
Table 28 Role-Based Access Control Algorithm	39
Table 29 External APIs	39
Table 30 System Testing.....	56
Table 31 Unit Testing 1: Authentication.....	57
Table 32 Unit Testing 2: Appointment Module.....	57
Table 33 Unit Testing 3: Medicine Store	57
Table 34 Unit Testing 4: Prescription Upload.....	57
Table 35 Unit Testing 5: AI Prediction Module.....	58
Table 36 Unit Testing 6: Admin Panel	58
Table 37 Automated Testing.....	61

Chapter 1

Introduction

Introduction

Curify is an AI-powered healthcare mobile application developed using Android Studio to enhance access to medical services in Pakistan by providing a unified platform for booking doctor appointments, purchasing medicines online, and managing prescriptions. The app integrates machine learning to enable features such as symptom-based disease prediction, health risk assessments, and personalized healthcare recommendations. Firebase is used for authentication and real-time data management, ensuring secure and efficient backend operations. Following the Agile methodology, the project has been developed in iterative phases and currently includes modules for user management, doctor appointment scheduling, virtual medical store, medicine management, predictive analytics, notifications, payment integration, and administrative control, with delivery tracking planned as a future enhancement.

1.1 Brief

Curify is an AI-powered healthcare mobile application developed to enhance access to medical services in Pakistan. The outcome of this work is a unified platform built using Android Studio and Firebase, which integrates features for booking doctor appointments, purchasing medicines online, and managing prescriptions. Machine learning modules provide symptom-based disease prediction and personalized recommendations. This report details the Agile methodology used, the system's design and architecture, and the implementation of its core features.

1.2 Relevance to Course Modules

This project is directly related to several core courses of the BCS program:

- **Software Engineering & Software Design & Architecture:** Guided the use of UML diagrams, SDLC models (Agile), and the creation of the SDD.
- **Object-Oriented Programming:** The entire system is designed and implemented using OOP principles in Kotlin.
- **Database Systems:** Firebase Realtime Database and Firestore are used for data management, design, and querying.
- **Artificial Intelligence:** The CureAI module employs ML for symptom checking and health risk assessments.
- **Human-Computer Interaction:** Principles of HCI were applied in designing the intuitive and user-friendly interface.

1.3 Project Background

The healthcare sector in Pakistan faces challenges like inaccessibility, especially in rural areas, and a lack of integrated digital services. Current solutions often consist of disparate platforms for appointments, pharmacy, and information. Curify addresses this by creating a centralized, intelligent platform that bridges the gap between patients, doctors, and pharmacies, leveraging mobile technology and AI to streamline healthcare delivery.

1.4 Literature Review

The global mobile health (mHealth) market is rapidly expanding. Existing applications like "Sehat Kahani," "Marham," and "Oladoc" in Pakistan focus primarily on teleconsultation and appointment booking. International platforms like "Practo" offer a more integrated suite. A key trend is the integration of AI for preliminary diagnoses, as seen in apps like "Ada Health."

However, a fully integrated solution combining AI-driven diagnostics, a full-fledged medical store, and surgery scheduling within a single, localized Pakistani app remains an innovative concept.

1.5 Analysis from Literature Review

While existing apps successfully digitize appointment booking, Curify differentiates itself by offering a more comprehensive ecosystem. Unlike "Marham" or "Oladoc," Curify includes an in-app medical store with inventory management and a predictive AI chatbot (CureAI). Our approach to integrate appointment booking, medicine ordering, prescription management, and AI-driven insights into a single platform provides a more holistic solution, aiming to become a one-stop shop for a user's healthcare needs.

1.6 Methodology and Software Lifecycle for This Project

The project follows the **Object-Oriented Design (OOD)** methodology and the **Agile Development Model**.

Rationale behind Selected Methodology

OOD was selected because Curify consists of various interacting entities (Users, Doctors, Appointments, etc.) that are naturally modeled as objects. This promotes code reusability, maintainability, and scalability. The **Agile model** was chosen for its iterative and flexible approach, allowing the team to develop the application incrementally, incorporate user feedback, and adapt to changing requirements, which is crucial for a feature-rich app like Curify.

Chapter 2

Problem Definition

2 Problem Definition

This chapter precisely defines the problem Curify aims to solve and outlines the project's deliverables.

2.1 Problem Statement

The Pakistani healthcare system lacks a unified, digital platform that provides integrated access to appointment scheduling, medicine purchasing, and AI-powered preliminary health assessments. This leads to fragmented services, time-consuming processes for patients, and inefficient resource management for doctors and pharmacies.

2.2 Deliverables and Development Requirements

- A fully functional Android application (.APK file).
- A comprehensive Project Report and SDD document.
- A deployed Firebase backend with secure authentication and real-time database.
- Source code developed in Kotlin using Android Studio.
- Integrated payment gateways (Easypaisa/JazzCash).
- An AI-based symptom checker module (CureAI).

2.3 Current System

The current system relies on manual processes: physically visiting clinics for appointments, purchasing medicines from physical pharmacies, and having no integrated digital platform for prescription management or AI-driven health insights. This is inefficient and time-consuming.

Chapter 3

Requirement Analysis

3 Requirement Analysis

Curify is a mobile healthcare application designed to improve access to medical services in Pakistan by offering a centralized platform for doctor appointments, online medicine purchases, surgery scheduling, and health monitoring.

3.1 Detailed Use Case

The table below indicates a comprehensive use case template filled in with an example drawn from the Medical Application.

Table 1 Login

Use case id	01
Use case name	Login
Actors	Admin, Customer, Doctor
Description	A user of the System logs into the System.
Trigger	User requests to log in.
Pre-condition	The user has an account.
Postcondition	The user is successfully logged into the system.
Normal flow	• User enters email/phone number and password. • System validates the credentials. • User is granted access to their respective dashboard.
Alternative flow	• If credentials are invalid, an error message is displayed. • User is prompted to retry or reset their password.
Exceptions	• Incorrect credentials. • Request to re-enter the login details. • Network failure or server downtime.
Business rules	The system must be connected to the network.

Table 2 Search for Doctors

Use case id	02
Use case name	Search for Doctors
Actors	Customer

Description	Customers search for doctors by specialty, location, or availability.
Trigger	User initiates a search request.
Pre-condition	Doctors are registered in the system.
Postcondition	Search results are displayed.
Normal flow	• User enters search criteria (e.g., specialty, location). • System fetches and displays relevant doctors.
Alternative flow	No doctors match the search criteria; system displays a "No results found" message.
Exceptions	No doctors match the search criteria; system displays a "No results found" message.
Business rules	Search must be efficient and responsive.

Table 3 Book an Appointment

Use case id	03
Use case name	Book an Appointment
Actors	Customer
Description	Customers book an appointment with a doctor.
Trigger	User selects a doctor and requests an appointment.
Pre-condition	Doctor's availability is updated in the system.
Postcondition	Appointment is successfully booked.
Normal flow	• User selects a doctor. • User chooses a date and time from the available slots. • System confirms the booking and sends notifications to both parties.
Alternative flow	Chosen slot is no longer available; user is prompted to select another slot.
Exceptions	Payment failure (if applicable) or system crash.
Business rules	Appointment data must be synchronized in real-time.

Table 4 Order Medicines

Use case id	04
Use case name	Order Medicines
Actors	Customer, Admin

Description	Customers place orders for medicines.
Trigger	User selects medicines and initiates an order.
Precondition	Medicines are available in stock.
Postcondition	Order is successfully placed.
Normal flow	• User browses or searches for medicines. • User adds medicines to the cart. • User confirms the order and makes payment. • System notifies the Admin to process the order.
Alternative flow	Medicines are out of stock; Admin notifies the user.
Exceptions	Payment gateway errors or inventory mismatch.
Business rules	Prescription validation is required for certain medicines.

Table 5 Medicine Details using AI

Use case id	05
Use case name	Medicine Details
Actors	Customer
Description	Customers inquire about medicine details, and the AI provides relevant information.
Trigger	User searches or selects a medicine to get details from the AI.
Precondition	Medicines are available in stock, and AI is active to provide information.
Postcondition	The customer receives detailed information about the medicine, or the system alerts the user if there is an issue.
Normal flow	1. User searches for medicine via the AI-powered system. User adds medicines to the cart. 2. AI provides detailed medicine information (e.g., description, dosage, side effects). 3. User adds medicine to the cart if interested. 4. User confirms order and proceeds to payment. 5. System notifies the Admin to process the order.
Alternative flow	AI notifies the user that the medicine is out of stock and suggests alternative medicines based on similar treatments or categories
Exceptions	AI Chat Bot is unavailable due to server error or some other issues
Business rules	AI checks if the selected medicine requires a prescription and prompts the user to upload a valid prescription before proceeding with the order.

Table 6 Manage Appointments

Use case id	06
Use case name	Manage Appointments
Actors	Doctor
Description	Doctors manage their appointments.
Trigger	Doctor logs into their dashboard.
Pre-condition	Appointments are booked in the system.
Postcondition	Appointments are updated or confirmed.
Normal flow	1. Doctor views the list of appointments. 2. Doctor confirms or reschedules appointments. 3. System notifies the customer about any updates.
Alternative flow	Doctor cancels an appointment; system notifies the customer and updates availability.
Exceptions	Synchronization issues with the customer's dashboard.
Business rules	Doctors must confirm changes promptly.

Table 7 Process Medicine Orders

Use case id	07
Use case name	Process Medicine Orders
Actors	Admin
Description	Admin process and dispatch medicine orders.
Trigger	Admin receives a new order notification.
Pre-condition	Orders are placed by customers.
Postcondition	Medicines are dispatched.
Normal flow	1. Admin reviews the order details. 2. Admin prepares the order for dispatch. 3. System updates order status and notifies the customer.
Alternative flow	Medicines are unavailable; Admin marks the order as "Out of Stock"
Exceptions	Order cancellation by the customer or Admin.
Business rules	Admin must verify prescription validity before processing.

Table 8 Cancel appointments

Use case id	08
-------------	----

Use case name	Cancel appointments
Actors	Customer, Doctor
Description	Customers and Doctor both can cancel bookings.
Trigger	Customers want to cancel the booking. Doctor cancels booking due to unavailability.
Precondition	The customer must be registered. Doctor must be registered.
Postcondition	Doctor can cancel the booking. Customer can cancel bookings after login into their account.
Normal flow	The customer successfully logs in to the system. The customer successfully cancels bookings.
Alternative flow	Admin does not cancel the bookings. The customer does not cancel the bookings until the admin accepts this request.
Exceptions	There is no internet. The error occurs in the application. Server down
Business rules	The request is rejected if the customer does not cancel his appointment within 2 days.

Table 9 Appointment Booking Payment

Use case id	09
Use case name	Appointment Booking Payment
Actors	Customer, Doctor, Admin
Description	After booking an appointment, the customer selects the payment method and confirms payment. Admin manages the payment, and sends a confirmation email to the customer.
Trigger	Customer wants to confirm the payment for the appointment. Admin manages the payment.
Precondition	The customer must be registered and must have an active appointment.
Postcondition	The customer can view the confirmed appointment payment and receives a confirmation email. Admin updates the appointment booking table.
Normal flow	The customer successfully logs in to the system, selects the payment method, confirms the payment, and the system processes the payment.
Alternative flow	The customer does not log in or does not select a payment method.
Exceptions	No internet connection, error in the application, or server issues preventing payment processing.
Business rules	The customer can only confirm payment after booking an appointment.

Table 10 Delivery Tracking (Future Module)

Use case id	10
Use case name	Delivery Tracking
Actors	Customer, Delivery Personnel
Description	Customers track the delivery of their orders.
Trigger	Order is dispatched.
Precondition	Delivery personnel are assigned to the order.
Postcondition	Order status is updated as delivered.
Normal flow	1. Delivery personnel updates the delivery status in real-time. 2. Customer views the delivery progress on the app. 3. Delivery is completed, and the system sends a confirmation.
Alternative flow	Delivery is delayed; system notifies the customer.
Exceptions	Network issues or GPS tracking failure.
Business rules	Secure verification is required for prescription medicine delivery

3.2 Functional Requirements

These are the following modules that will be included in this system.

1. Customer Module
2. Doctor Module
3. Medical Store Module
4. Admin Module
5. Future Module: Delivery personnel

Table 11 Customer's Module Functionalities

Identifier	01
Title	Customer's Module Functionalities
Requirement	This module is designed to provide patients/customers with a platform for medical consultations and ordering medicines.
Source	Business need for a user-friendly platform for healthcare services.
Rationale	To enable seamless access to medical consultations, medicine ordering, and secure transactions for customers.
Business Rule (if required)	The system must ensure secure handling of payment details and maintain confidentiality of customer information.

Dependencies	Integration with payment gateway, doctor's module, and inventory system for medicine orders.
Priority	High

Table 12 Doctor's Module Functionalities

Identifier	02
Title	Doctor's Module Functionalities
Requirement	This module is designed to enable doctors to provide consultations and medical advice while managing their profiles and schedules.
Source	Need for a system to manage doctor-patient interactions efficiently.
Rationale	To provide doctors with tools to manage patient consultations, prescribe medications, and maintain health records securely.
Business Rule (if required)	Only verified delivery personnel can access and fulfill orders. medicines require secure verification at the point of delivery.
Dependencies	Integration with the customer module, health records database, and payment gateway for consultation fees.
Priority	High

Table 13 Medical Store Module Functionalities

Identifier	03
Title	Medical Store Module Functionalities
Requirement	This module is designed for admins to manage the online medical store, including medicines, orders, inventory, and payments.
Source	Need for an efficient system to manage medicine inventory, orders, and secure payment processing.
Rationale	To ensure the availability of medicines, streamline order processing, and manage inventory effectively.
Business Rule (if required)	Only authorized admins can add/remove medicines or manage inventory. Payment processing must comply with security standards.
Dependencies	Integration with customer module, inventory management system, and payment gateway.
Priority	High

Table 14 Admin Module Functionalities

Identifier	04
Title	Admin Module Functionalities
Requirement	This module is designed for administrators to manage the system, including medicines, inventory, orders, and payments.
Source	Need for centralized control over the system for efficient management and operation.

Rationale	To provide admins with the ability to maintain and manage the system, ensuring smooth operations and secure transactions.
Business Rule (if required)	Only authorized admin users can access and perform management operations. Payment processing must comply with security and privacy standards.
Dependencies	Integration with medical store module, customer module, and payment gateway.
Priority	High

Table 15 Delivery Personnel Module Functionalities

Identifier	05
Title	Delivery Personnel Module Functionalities
Requirement	This future module will enable delivery personnel to fulfill medicine orders and allow customers to track their deliveries in real time.
Source	Future need for integrating a delivery system to ensure seamless order fulfillment and tracking.
Rationale	To provide an efficient delivery mechanism for orders and enhance customer satisfaction through real-time tracking and secure delivery verification.
Business Rule (if required)	. Only verified delivery personnel can access and fulfill orders.
Dependencies	Integration with medical store module, customer module, and real-time tracking systems.
Priority	Medium (future implementation).

Table 16 AI Module

Identifier	06
Title	AI Chatbot Assistant Module
Requirement	This module will provide an AI-powered chatbot for customers and delivery personnel to handle queries, track orders, update statuses, verify deliveries, and offer real-time support across the system.
Source	Future need for automated assistance to reduce manual load, support real-time communication, and enhance customer & delivery personnel experience.
Rationale	To improve system usability, provide instant query resolution, assist delivery personnel in fulfilling orders, and enable customers to efficiently track and manage their deliveries.
Business Rule (if required)	Only authenticated users (customer or delivery personnel) can access personalized data such as order status, location, or delivery details. - AI assistant must not change any order status without user confirmation.
Dependencies	Integration with: customer module, delivery personnel module, order management module, real-time tracking system, notification service (SMS/push).
Priority	High (core enhancement improving usability and automation).

3.3 Non-Functional Requirements

Table 17 Non Functional Requirement

Category	Requirement	Specification	Priority
Performance	Response Time	App screens load within 2 seconds; API responses within 1 second	High
Performance	Concurrent Users	Support for 1,000+ simultaneous users	Medium
Usability	User Interface	Intuitive design requiring minimal training; 95% user satisfaction	High
Usability	Accessibility	Support for visually impaired users with screen reader compatibility	Medium
Reliability	System Uptime	99.5% availability with minimal downtime	High
Reliability	Data Consistency	Real-time data synchronization across all devices	High
Security	Data Protection	End-to-end encryption for sensitive data; secure authentication	High
Security	Privacy Compliance	GDPR-compliant data handling and user privacy protection	High
Compatibility	Platform Support	Android API level 21+ (Android 5.0 and above)	High
Scalability	Future Growth	Modular architecture supporting easy feature additions	Medium
Maintainability	Code Quality	Well-documented, modular code following Kotlin best practices	Medium

Chapter 4

Design and Architecture

4 Design and Architecture

4.1 System Architecture

Curify follows a **client-server architecture**. The mobile app (client), built with Android Studio, communicates with the Firebase backend (server) for all data operations, including authentication, real-time database updates, and cloud storage. The architecture also integrates machine learning modules for predictive features.

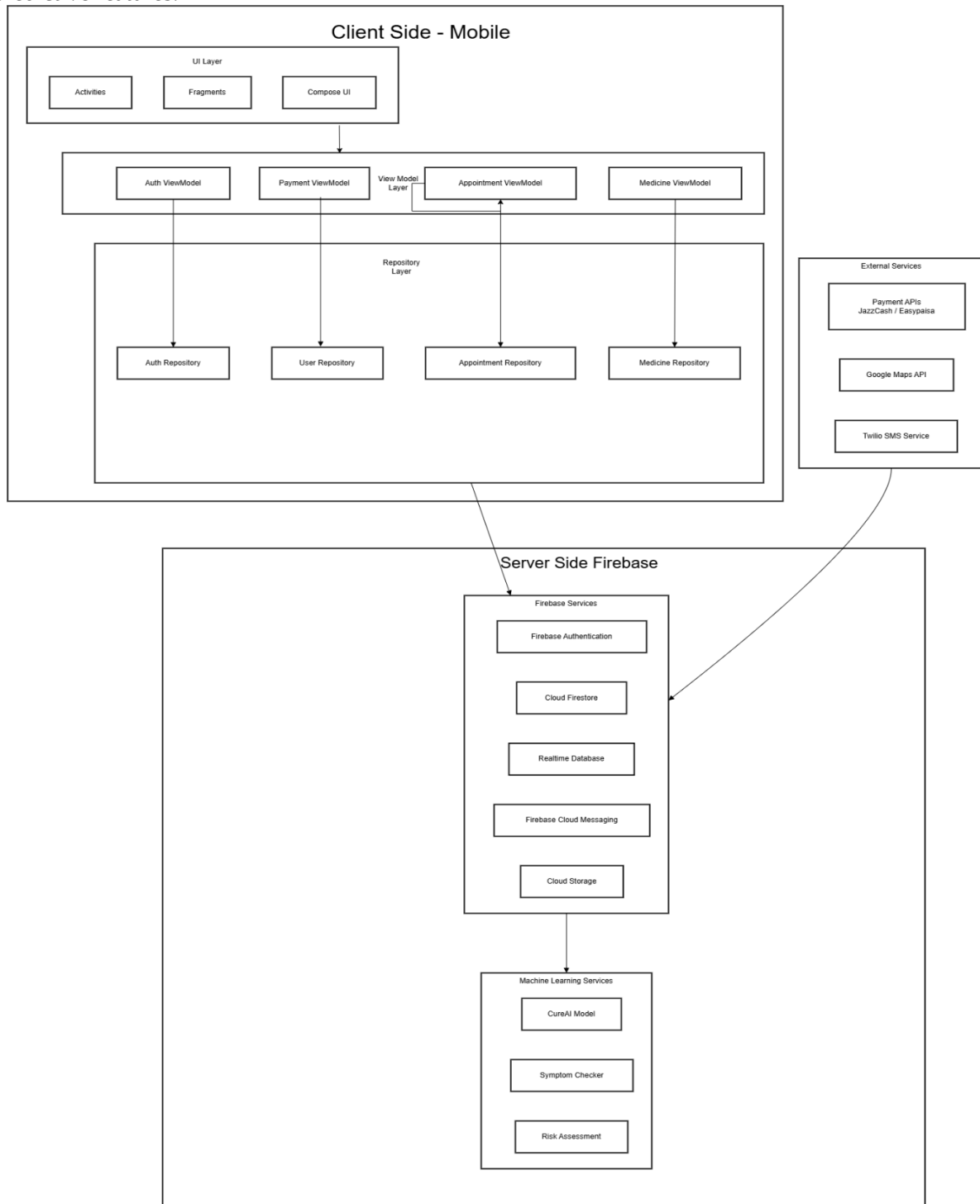


Figure 1 System Architecture

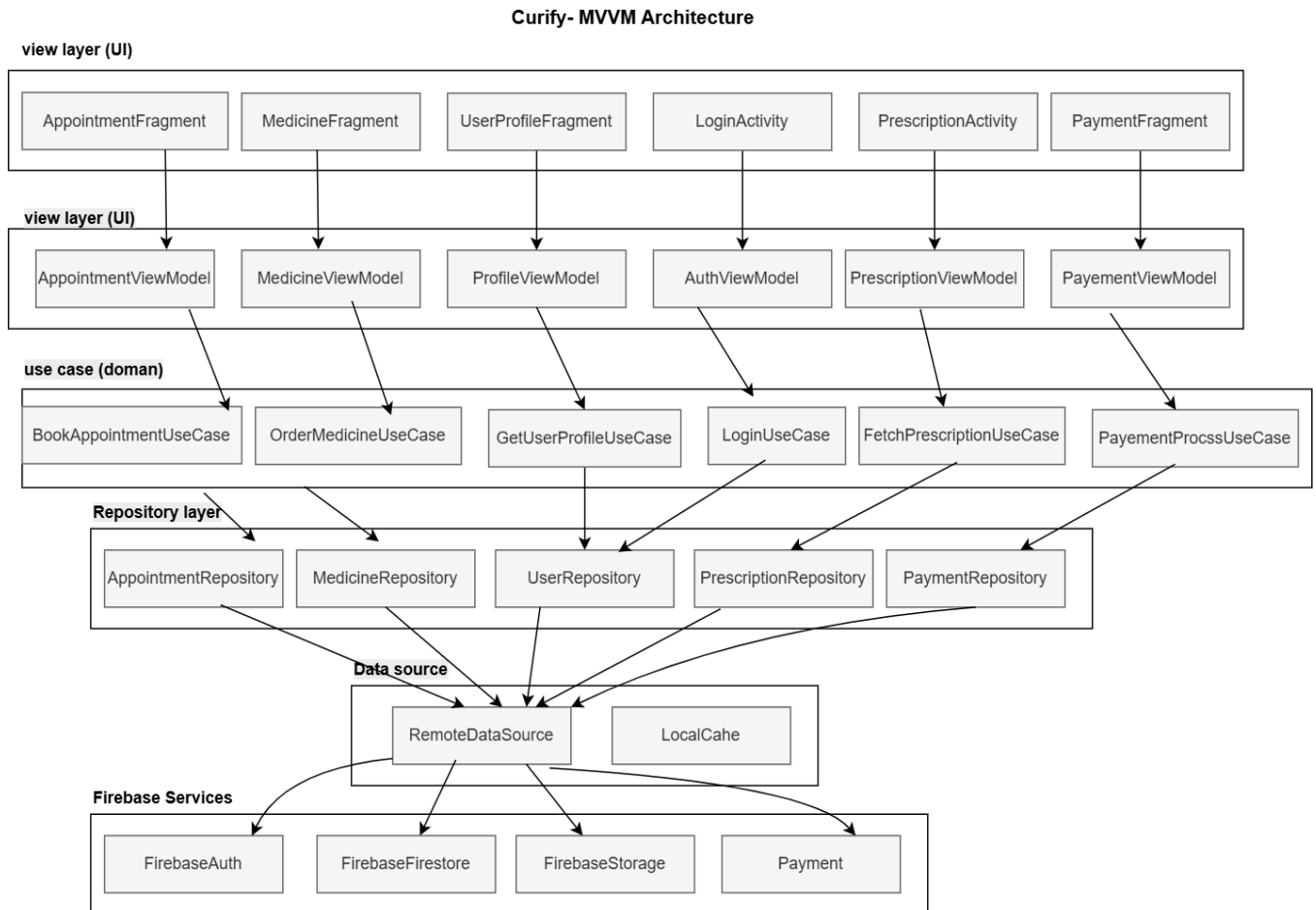


Figure 2 MVVM Architecture

Data Representation

The system's data is represented using an Object-Oriented model and stored in Firebase Firestore/Realtime DB. Major entities include User, Doctor, Appointment, Medicine, and Order. The **Data Dictionary** from the SDD (Section 5.1) provides a detailed description of these entities, their attributes, and methods.

Use Case for Curify

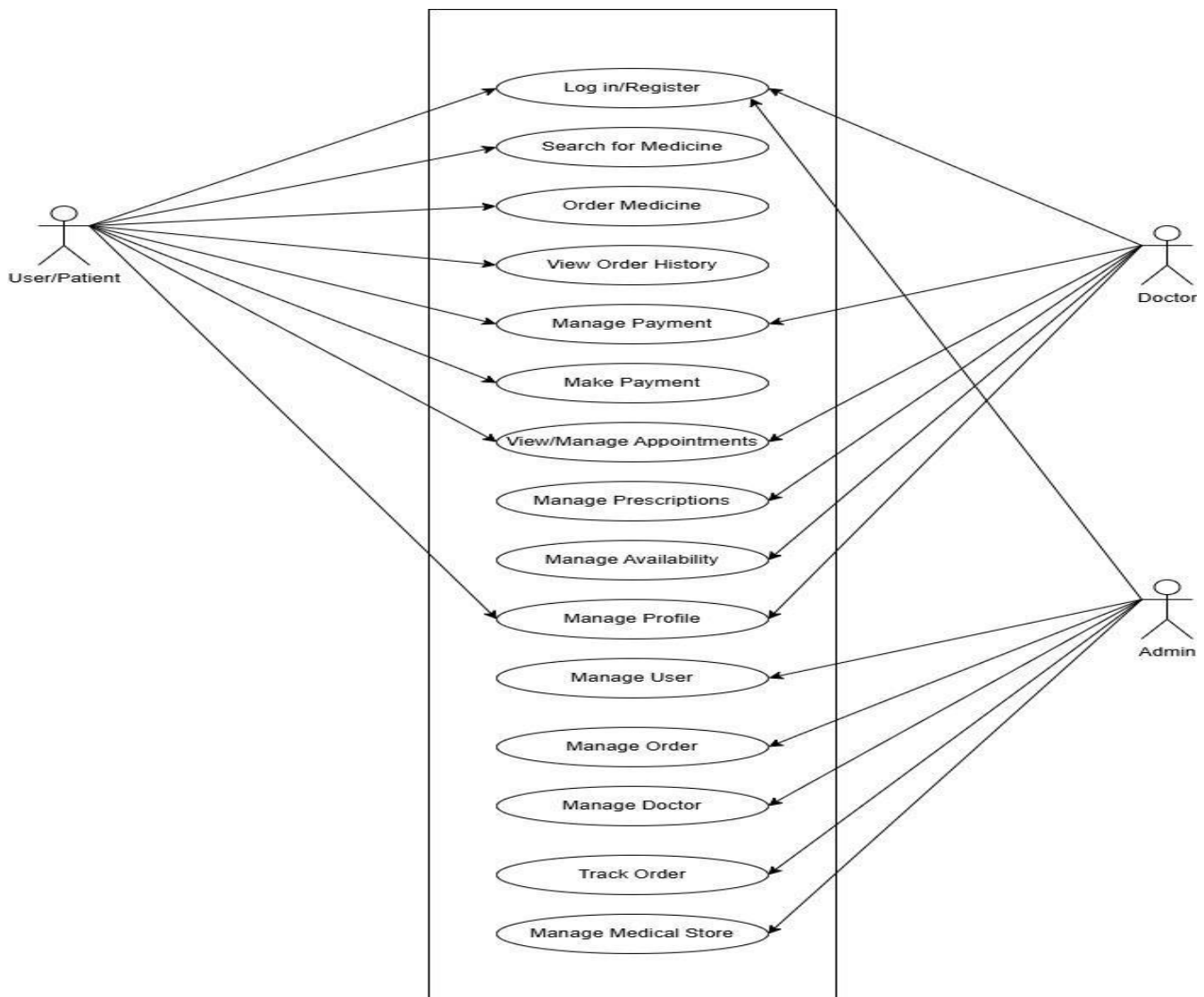


Figure 3 Use Case Diagram For Curify

This is a structural overview for a multi-role healthcare platform that integrates the functions of an online pharmacy (medicine search, ordering) with healthcare management (appointments, prescriptions) and a full administrative backend (managing users, doctors, stores, and orders). The design is in a very early, text-based state, focusing on functionality and information architecture rather than visual aesthetics.

4.2 Data Dictionary

Below is an alphabetically arranged data dictionary based on the object-oriented structure of the system:

Table 18 Data Dictionary

Entity/Object	Type	Attributes	Methods
Admin	Object	adminID, name, email, password, phone	manageUsers(), updateInventory(), viewReports()
Appointment	Object	appointmentID, patientID, doctorID, date, time, status	book(), cancel(), reschedule()
Doctor	Object	doctorID, name, email, phone, specialization, availability, password	approveAppointment(), updateAvailability()
Medicine	Object	medicineID, name, category, price, stock, description	updateStock(), checkAvailability()
Notification	Object	notificationID, userID, message, timestamp, type	sendNotification()
Order	Object	orderID, userID, medicineList[], totalAmount, status, paymentMethod, timestamp	placeOrder(), trackOrder(), cancelOrder()
Patient/User	Object	userID, name, email, phone, password, address, medicalHistory[]	bookAppointment(), orderMedicine(), checkSymptoms()
Symptom Checker	Function	symptomList[]	checkSymptoms()
Health Risk Assessment	Function	userData	assessRisk()

4.3 Process Flow/Representation

The **Activity Diagram** represents the high-level workflow, starting from user authentication and branching into core functionalities like booking appointments, ordering medicines, and making payments, with error handling and Firebase integration at each step.

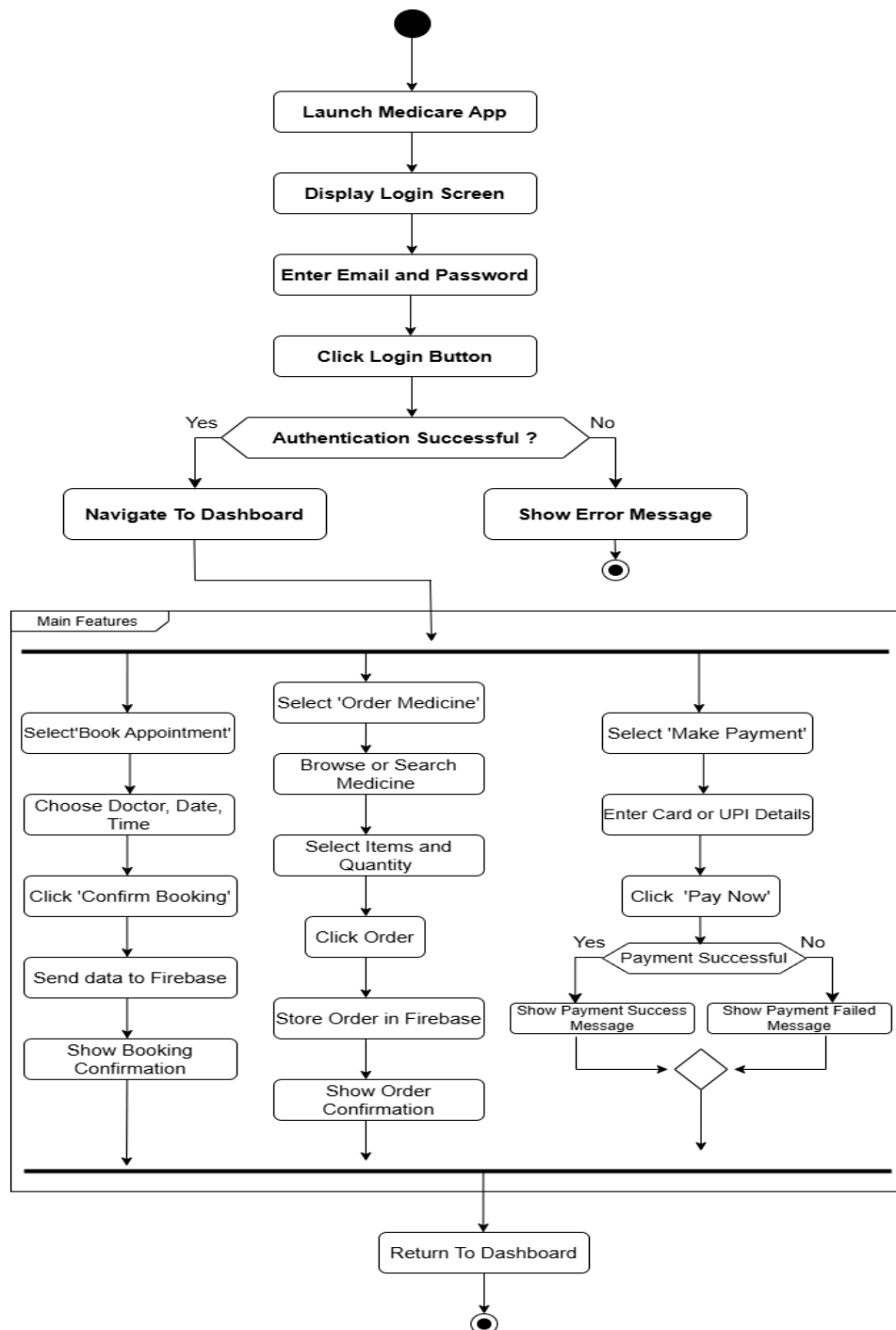


Figure 4 Activity Diagram

4.4 Design Models

This is important to include design models that provide a clear and comprehensive understanding of the system's structure and functionality.

4.4.1 Class Diagram

The attributes and methods of each class can be further defined based on your specific requirements and implementation details in the class diagram. The following class diagram is as followed:

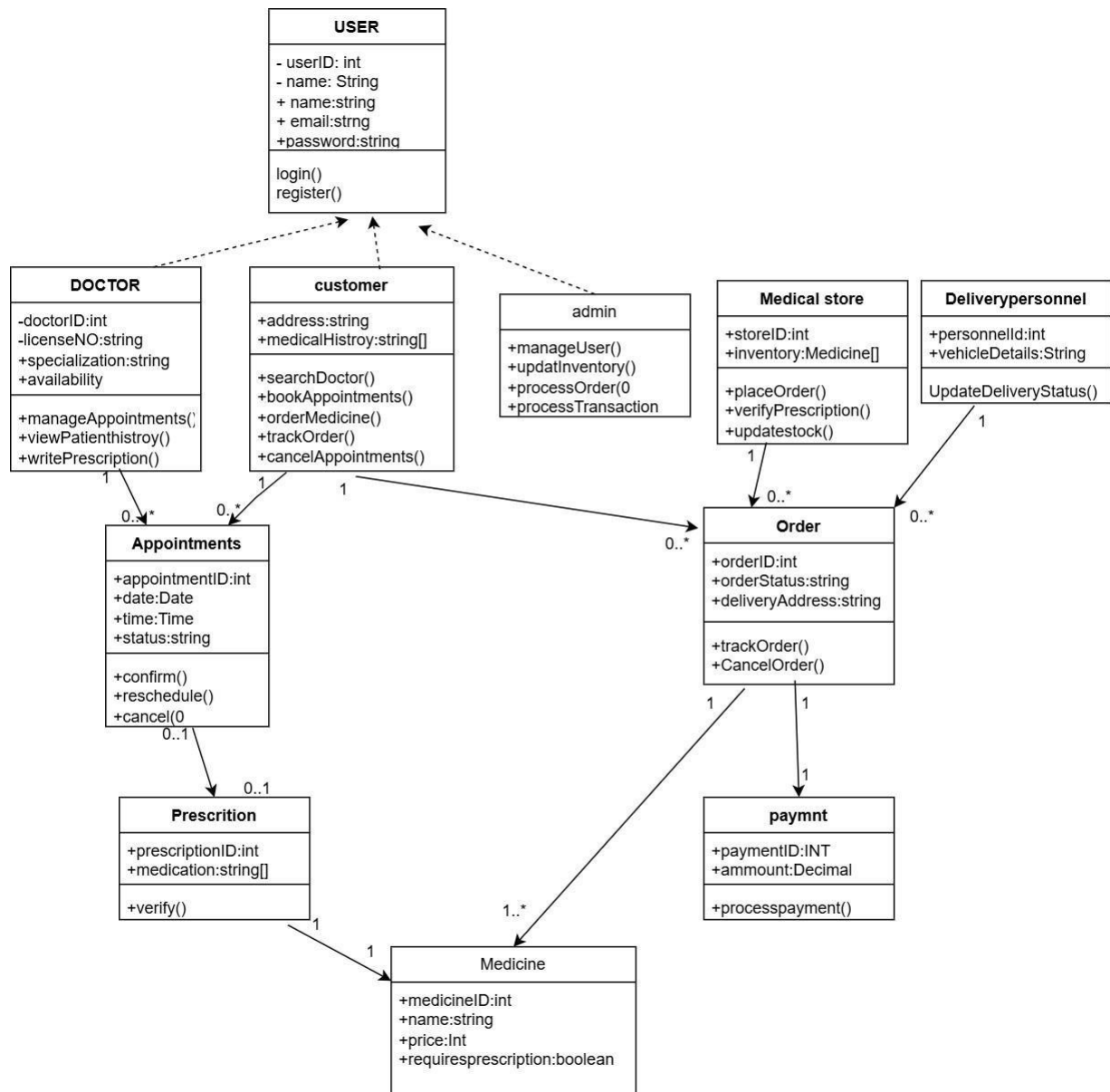


Figure 5 Class Diagram

4.4.2 ERD Diagram

The Curify database schema consists of nine core entities: USER, DOCTOR, and ADMIN handle different user roles; APPOINTMENT and PRESCRIPTION manage healthcare services; MEDICINE, ORDER, and ORDER_ITEM handle pharmacy operations; while PAYMENT and NOTIFICATION support transactions and communications. All entities are linked through key relationships - users can have multiple appointments, orders, and prescriptions; doctors manage appointments and prescriptions; orders contain multiple medicine items; and each order has a corresponding payment record, creating a complete healthcare management system.

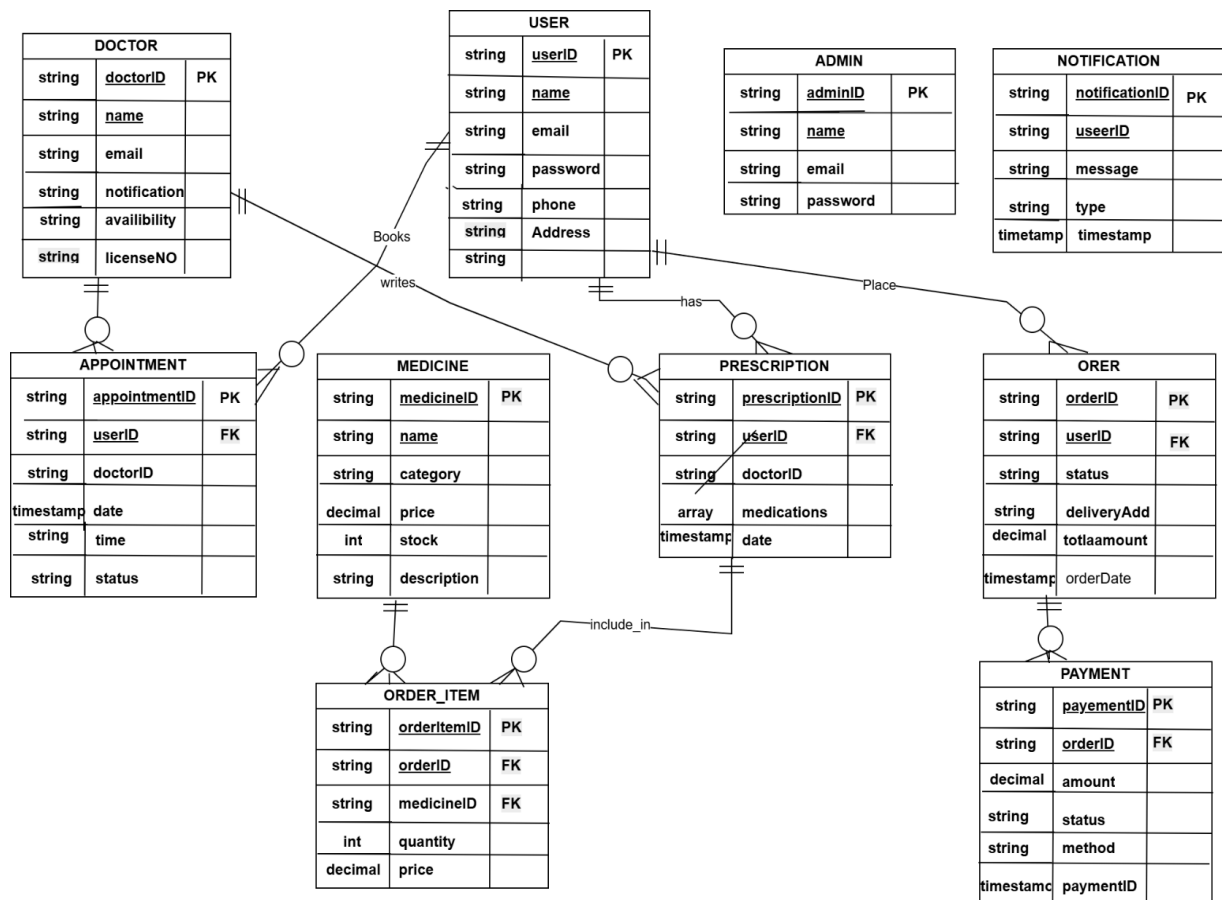


Figure 6 ERD Diagram

4.4.3 Sequence Diagram

The sequence diagrams illustrate the interaction flow between users (patients, doctors, admins) and system components during key operations such as booking appointments, ordering medicines, and processing payments. Each diagram represents the chronological exchange of messages between the mobile app, backend services (Firebase), and external modules like the payment gateway or AI Chatbot.

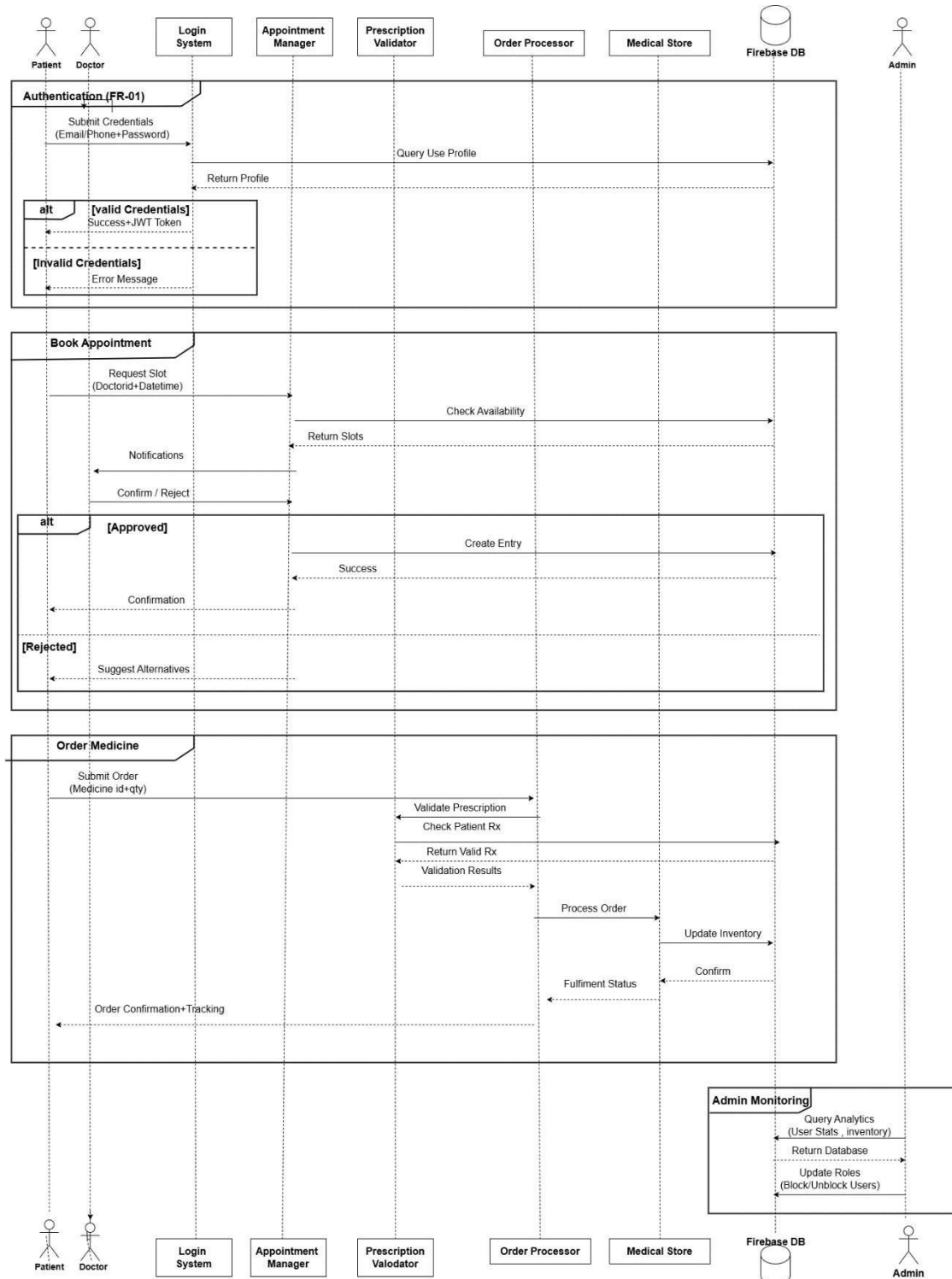


Figure 7 Sequence Diagram

4.4.4 State Transition Diagram

It starts at the splash screen (oval), then guides users through key actions like login and messaging (rectangles). Decision points (diamonds) check conditions, such as whether a user is logged in. Arrows show how users move between screens, while rounded rectangles mark the start and end of the flow. Connectors link related processes for a complete overview.

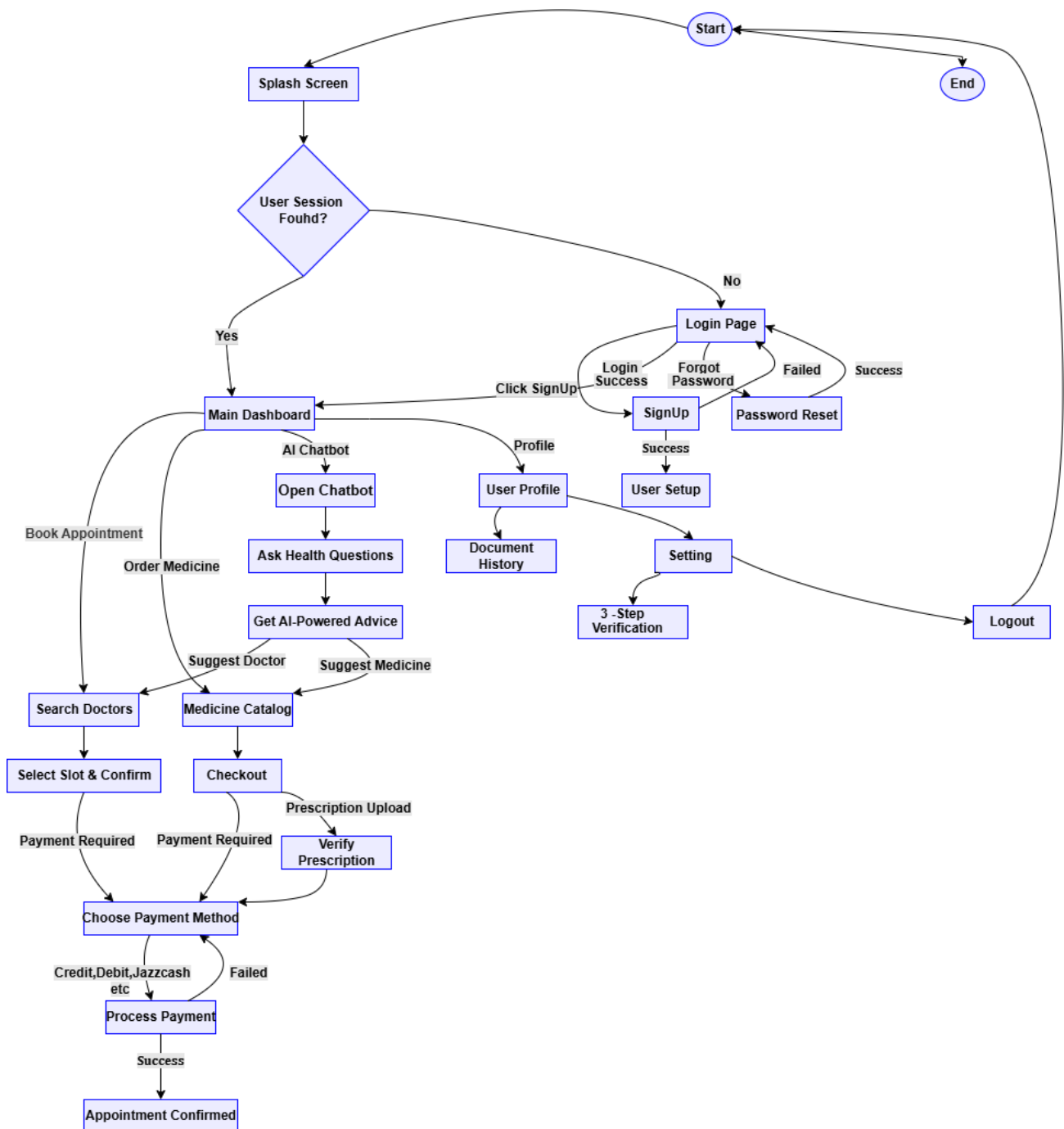


Figure 8 State Transition Diagram

4.4.5 Data Flow Diagram

The Level 0 Data Flow Diagram provides a high-level overview of the entire Curify system, representing it as a single process with its interactions with external entities such as users (patients, doctors, admins) and the Firebase database. It shows the major data flows between the system and external actors, including appointment requests, medicine orders, user authentication, and data retrieval. This diagram helps in understanding the core inputs and outputs of the system without diving into internal processes.

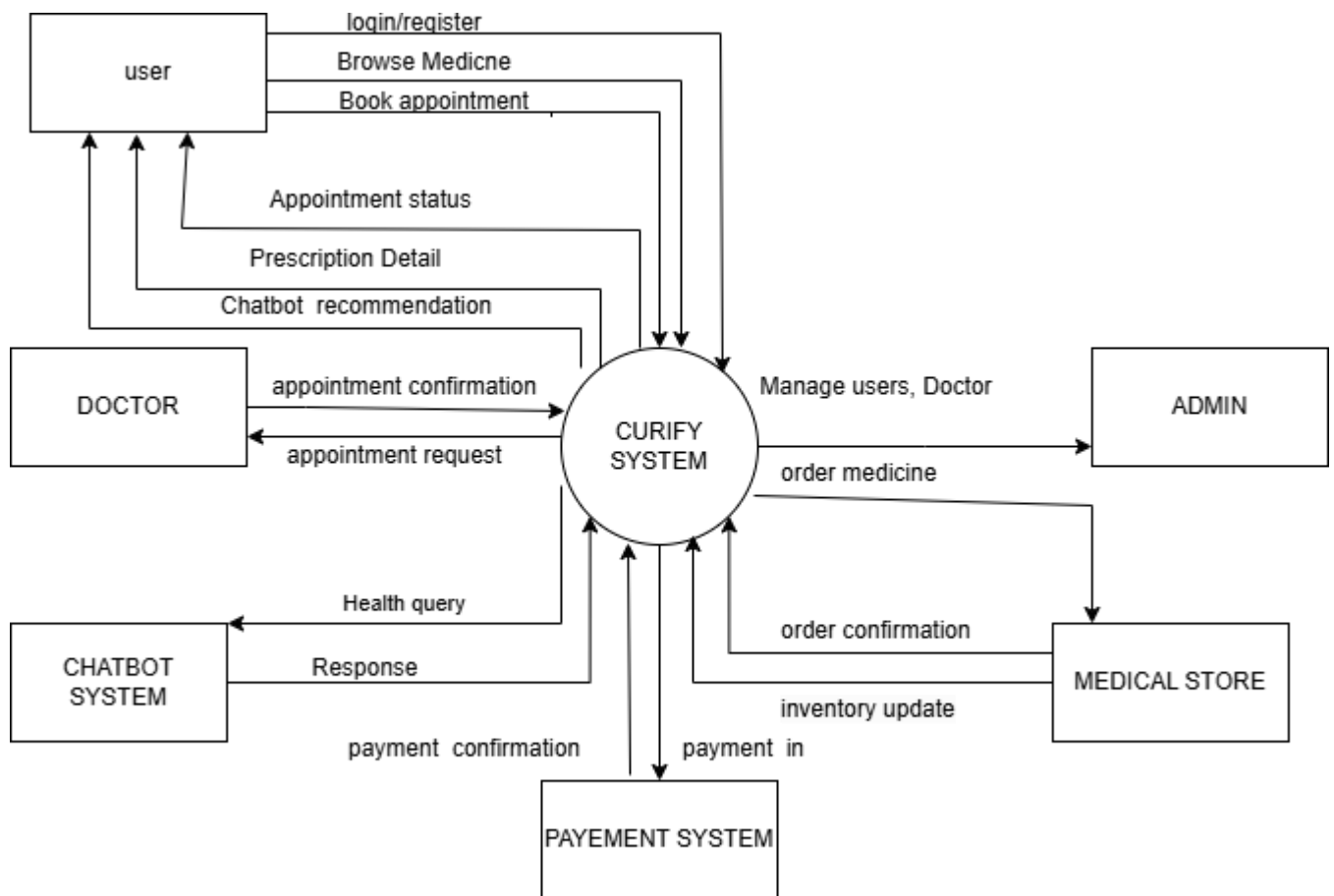


Figure 9 DFD Level 0

DFD Level 1

The Level 1 Data Flow Diagram breaks down the main system process into its major sub-processes, offering a more detailed view of how data moves within Curify. It includes processes such as user registration and login, appointment booking, medicine ordering, and admin management. Each sub-process interacts with relevant data stores and external entities, showing how user inputs are processed and how information flows between modules and the Firebase backend. This diagram provides a clearer understanding of the system's internal logic and data handling.

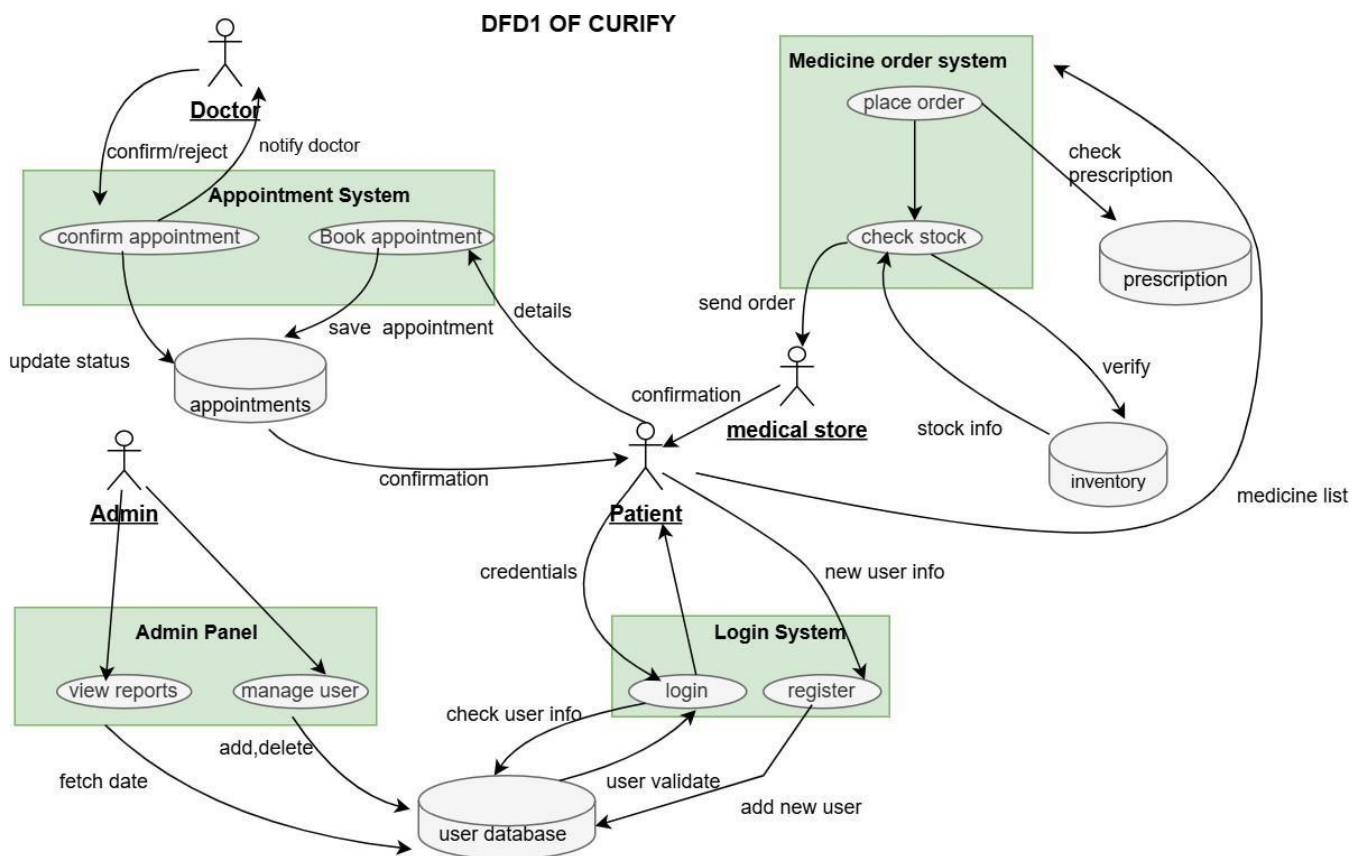


Figure 10 DFD Level 1

DFD Level 2

The Level 2 Data Flow Diagram provides a detailed breakdown of specific sub-processes shown in Level 1. It focuses on individual operations such as the step-by-step flow for booking an appointment, placing a medicine order, or verifying user credentials. Each process is further divided into smaller functional components, showing precise data interactions with the Firebase database, validation steps, and user notifications. This level of detail helps in understanding the internal workflows of core modules and supports effective system implementation and troubleshooting. The DFD also includes admin-specific processes such as managing user and system data through the "Admin Management" module. The integration with Firebase ensures secure data storage, while user logs are maintained for monitoring and audit purposes. An AI Chat Bot Assist module is included to enhance user experience by providing automated support. Clear data flows between external entities like Customer, Admin, and Payment Gateway highlight how the system handles requests and responses efficiently. Overall, the diagram offers a comprehensive view of how each functional area operates and interacts within the system architecture.

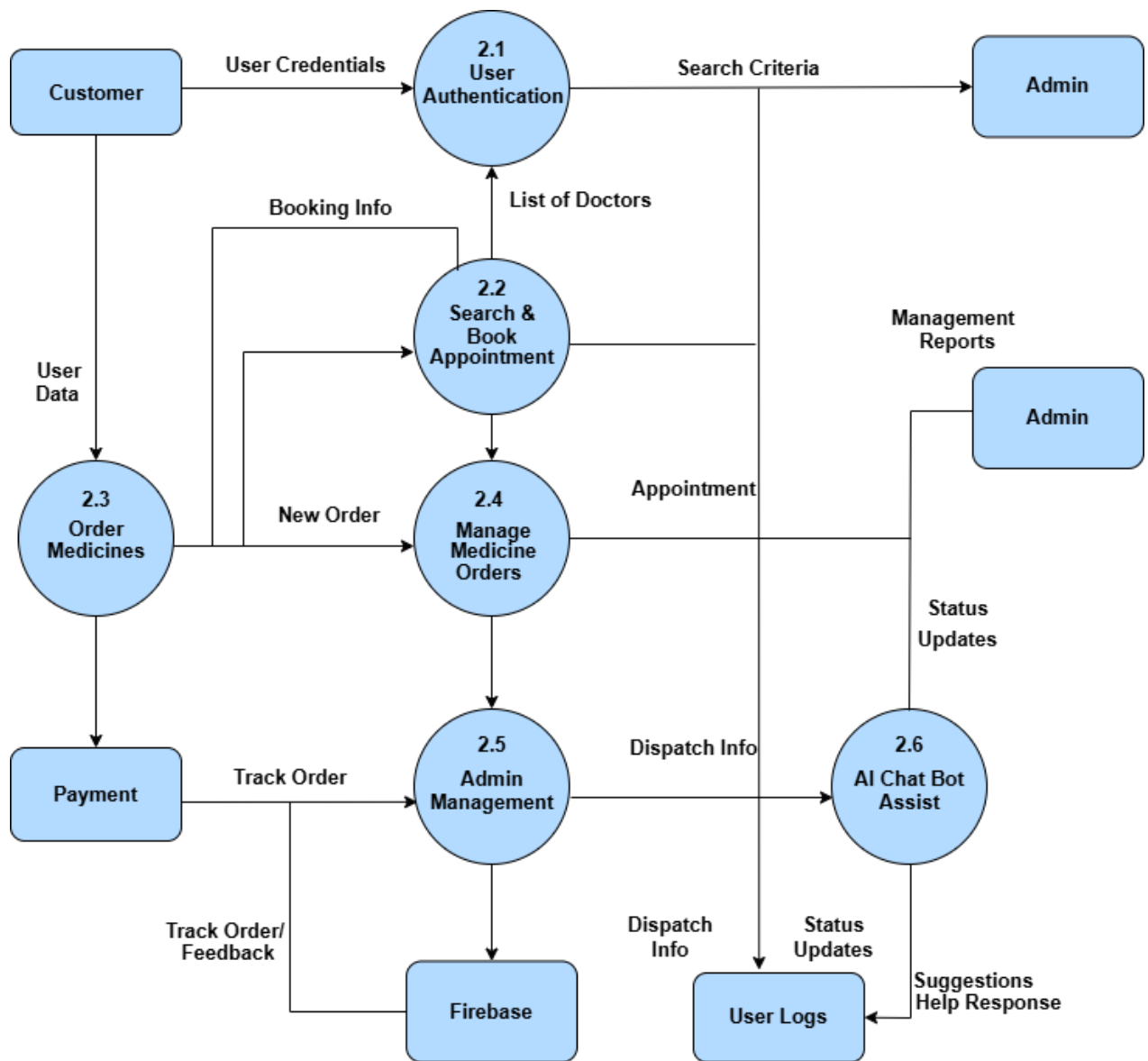


Figure 11 DFD Level 2

Chapter 5

Implementation

5 Implementation

The implementation phase focuses on converting the system design into an operational mobile application using Android Studio and Firebase. The Curify system has been developed using the Kotlin programming language for the frontend and Firebase for backend services, including Authentication, Firestore Database, Cloud Storage, and ML-based modules.

The implementation follows a modular approach, where each feature is developed as an independent module to ensure maintainability and easy debugging. The MVVM (Model–View–ViewModel) architecture divides the code into logical layers:

- Model Layer
- View Layer
- ViewModel Layer

The implementation includes:

1. User Authentication Module
2. Appointment Management Module
3. Medicine Store Module
4. Digital Prescription & Report Upload Module
5. AI-Based Disease Prediction Module
6. Verification & Notification Module
7. Chat/Support Module
8. Admin Workflow Management Module

5.1 Algorithms

This section describes the key algorithms implemented within the Curify system. Each algorithm outlines the step-by-step execution flow of major system operations, including authentication, appointment booking, medicine ordering, digital prescription management, disease prediction, and notification handling. The algorithms ensure that Curify performs efficiently, securely, and in accordance with healthcare application standards.

User Authentication Algorithm

Table 19 User Authentication Algorithm

Purpose: Manage	Securely verify user identity and manage access control
Workflow	Validates login credentials against Firebase Auth, retrieves user role from Firestore, and redirects to appropriate dashboard based on user type (Patient/Doctor/Admin)
Key Features	Role-based redirection, secure credential verification, error handling for invalid attempts

User enters email & password

System checks: if (newUser) → createUserInFirestore()

else → verifyCredentials()

If authentication successful:

fetchUserRole() from Firestore

redirect user to appropriate dashboard (Patient / Doctor / Admin)

Else:

showError("Invalid Credentials") and request re-entry

Appointment Booking Algorithm

Table 20 Appointment Booking Algorithm

Purpose: Manage	doctor appointment scheduling with real-time availability checks
Workflow	Checks doctor's schedule in Firestore, books available slots, updates database, and notifies doctor of new appointments
Key Features	Real-time slot availability, automatic schedule updates, instant doctor notifications

User selects doctor → selects available date & time

System fetches doctor schedule from Firestore

If selected slot is available:

createAppointmentEntry()

update doctor's schedule in Firestore

notifyDoctor("New Appointment Request")

Else:

show message: "Selected slot unavailable"

Digital Prescription Upload Algorithm

Table 21 Digital Prescription Upload Algorithm

Purpose: Manage	Handle secure upload and storage of medical prescriptions
Workflow	Compresses images/PDFs for optimization, stores files in Firebase Storage, and saves metadata with user/doctor information
Key Features	File compression, secure cloud storage, metadata tracking for audit trail

User selects prescription image/PDF

System compresses image for optimization

Upload file to Firebase Storage

On successful upload:

 retrieve downloadURL

 store metadata (userID, doctorID, timestamp) in Firestore

 return confirmation to user

Else:

 showError("Upload Failed")

Medicine Ordering Algorithm

Table 22 Medicine Ordering Algorithm

Purpose: Manage	Process medicine orders with inventory management
Workflow	Validates stock availability, calculates totals, creates orders, updates inventory levels, and notifies admins
Key Features	Real-time stock verification, automatic inventory deduction, admin notifications

User adds medicines to cart

System calculates totalAmount()

When user confirms order:

 verify medicine stock from Firestore

 if stockAvailable:

 createOrderEntry()

 deductStockFromInventory()

 notifyAdmin("New Order Received")

 else:

 showError("Selected item is out of stock")

AI-Based Disease Prediction Algorithm

Table 23 AI-Based Disease Prediction Algorithm

Purpose: Manage	Provide intelligent symptom analysis and disease prediction
Workflow	Processes user symptoms through ML model, generates predictions with severity levels, and stores results in medical history
Key Features	Machine learning integration, symptom analysis, medical history tracking

User inputs symptoms / uploads medical image

System preprocesses input (resize, normalize, tokenize)

Run trained ML model → generate prediction

Retrieve top probable diseases

Display prediction results with:

description, severity level, possible causes, suggestions

Store prediction report in:

userMedicalHistory() in Firestore

Doctor Verification & Approval Algorithm

Table 24 Doctor Verification & Approval Algorithm

Purpose: Manage	Manage doctor credential verification process
Workflow	Admins manually review license documents, update verification status, and notify doctors of approval/rejection
Key Features	Manual verification process, status tracking, notification system

Doctor registers account → submits license details

Admin retrieves document → manually reviews

If details valid:

update doctor.status = "Approved"

notifyDoctor("Account Verified")

Else:

update doctor.status = "Rejected"

notifyDoctor("Verification Failed")

Notification Handling Algorithm (FCM)

Table 25 Notification Handling Algorithm (FCM)

Purpose: Manage	Manage real-time push notifications across the platform
Workflow	Triggers notifications for various events, sends via FCM, and updates UI on user devices
Key Features	Real-time messaging, event-driven notifications, cross-platform delivery

Trigger event occurs (appointment request, approval, order update)

System generates notification payload

Send notification through Firebase Cloud Messaging (FCM)

On user device:

FCM listener receives notification

display alert → update UI accordingly

Order Tracking Algorithm

Table 26 Order Tracking Algorithm

Purpose: Manage	Provide end-to-end order status tracking
Workflow	Updates order status through various stages (Pending → Preparing → Out for Delivery → Delivered) with user notifications
Key Features	Status lifecycle management, real-time updates, user notifications

User places order → system sets status="Pending"

Admin processes order → update status="Preparing"

Once shipped:

 update status="Out for Delivery"

 notifyUser("Your order is on the way")

When delivered:

 update status="Delivered"

 close order tracking loop

Chat / Support Algorithm

Table 27 Chat / Support Algorithm

Purpose: Manage	Enable real-time messaging between users and support
Workflow	Manages chat rooms in Firestore, handles message sending/receiving with real-time updates
Key Features	Real-time messaging, Firestore synchronization, chat room management

User opens chat screen

System loads messages in real-time from Firestore

When user sends a message:

 createMessageEntry()

 push to Firestore in chatRoom collection

 receiver's device triggers snapshot listener → displays new message

Role-Based Access Control Algorithm

Table 28 Role-Based Access Control Algorithm

Purpose: Manage	Enforce security through role-based permissions
Workflow	Determines user access rights based on role (Admin/Doctor/Patient) and restricts unauthorized module access
Key Features	Security enforcement, role-based permissions, access control

On login → fetch user.role from Firestore

If role == "Admin":

allow access to all management modules

If role == "Doctor":

allow access to appointments + prescriptions

If role == "Patient":

allow access to booking + ordering modules

Else:

deny inappropriate access

5.2 External APIs

This section provides the details of all external APIs used in the Curify mobile application. These APIs support essential functionalities such as authentication, cloud storage, real-time data handling, disease prediction, and map-based features. Each API is listed with its description, purpose, and the modules where it is integrated within the system.

Table 29 External APIs

Name of API	Description of API	Purpose of Usage	Function/Class Name Where Used
Firebase Authentication API	Provides secure user authentication using email/password.	To register users, login users, and handle secure access based on roles (patient, doctor, admin).	AuthRepository, LoginActivity, RegisterActivity
Firebase Firestore API	Cloud NoSQL database for structured real-time data.	To store user profiles, appointments, medicines, orders, and doctor details.	UserRepository, DoctorRepository, AppointmentRepository, OrderRepository

Firestore Cloud Storage API	Stores images, prescriptions, reports, and medicine photos.	To upload and retrieve user prescriptions, doctor verification docs, and product images.	StorageService, PrescriptionActivity, ProfileActivity
Firestore Cloud Messaging (FCM)	Push notification service for Android.	To send alerts for appointment updates, order status, admin approvals, and new messages.	NotificationService, FirebaseMessagingService
DeepSeek API (Replaced ML Kit + TFLite)	Cloud-based AI model used for NLP and reasoning.	To analyze symptoms, generate medical insights, AI chatbot responses, and disease prediction.	DeepSeekService, ChatBotViewModel, PredictionActivity
Google Maps API (Future Use)	Provides location-based map and navigation services.	For order delivery tracking and nearby doctor/pharmacy search.	DeliveryTrackingActivity, MapsService
Glide Image Loading API	Efficient image loading and caching library.	To display doctor images, medicine photos, and uploaded reports.	Glide.with(), ImageHelper, MedicineAdapter
Retrofit API	REST API client for external services.	To fetch medical data, interact with backend, or integrate external APIs.	ApiClient, MedicalApiService
SafePay Payment API (Added New)	Secure online payment processing for apps.	To handle medicine order payments, appointment payments, or subscriptions.	PaymentService, CheckoutActivity, PaymentRepository

5.3 User Interface

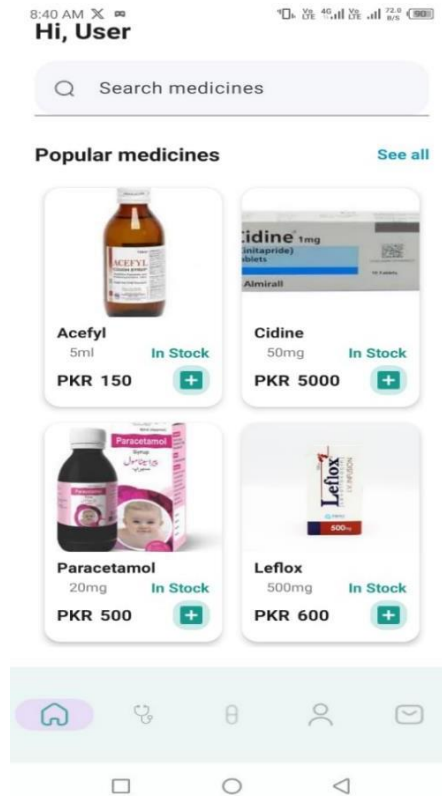


Figure 12 Medicine Shop

This shows the pharmacy section displaying popular medicines (Acetyl, Paracetamol, Leflox) with their dosages, stock status, and prices. It represents the medical store interface where users can browse and purchase medications.

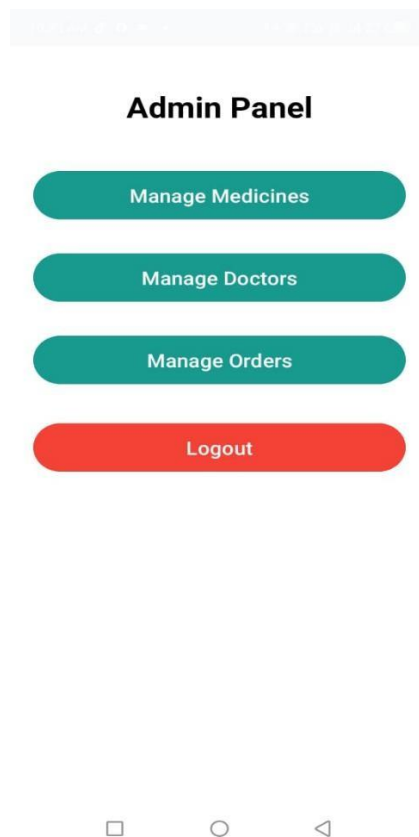


Figure 13 Admin Panel

This is the administrator interface with management options for Medicines, Doctors, and Users, plus Logout. It shows the backend control system for maintaining the platform's content and user base.

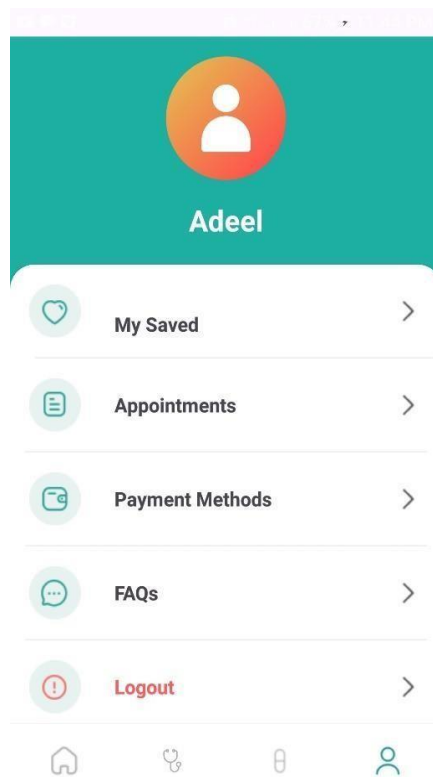


Figure 14 User Profile

This is a patient's profile screen showing navigation options: Saved items, Appointment history, Payment methods, FAQs, and Logout. It represents the main user dashboard for managing personal healthcare activities and account settings.

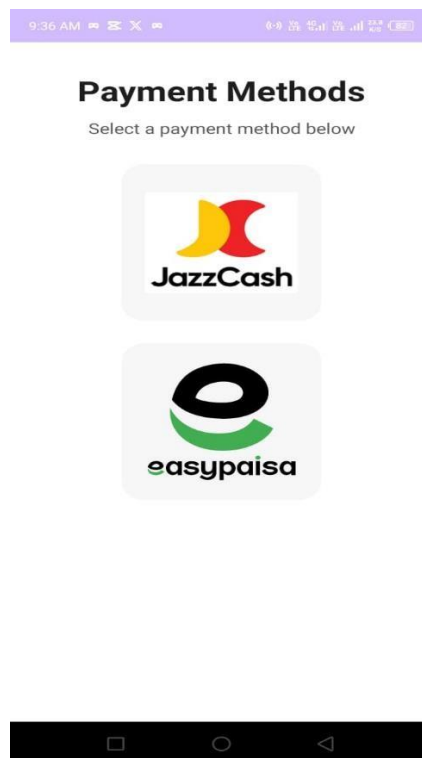


Figure 15 Payment Method

This screen presents payment options with JazzCash and Easypaisa gateways, allowing users to choose their preferred payment method for completing transactions on the platform

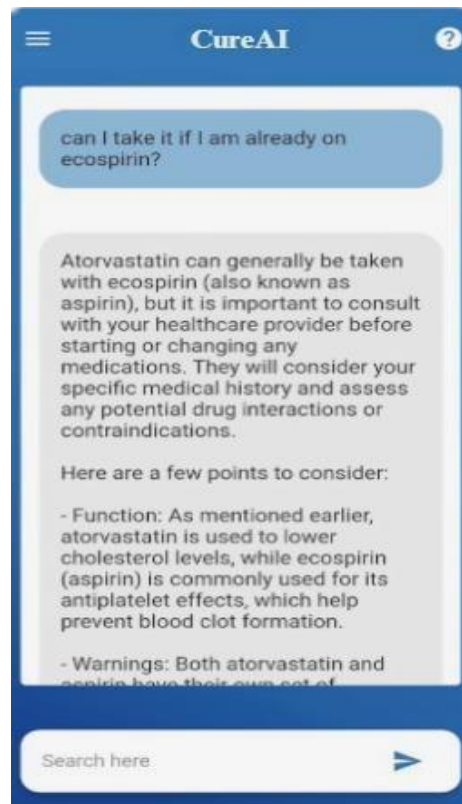


Figure 16 CureAI

This shows the CureAI feature in action - a user asked about drug interactions between Atorvastatin and Ecospirin. The AI provides a detailed, medically-informed response about compatibility, functions, and warnings, demonstrating the app's healthcare advisory capability.

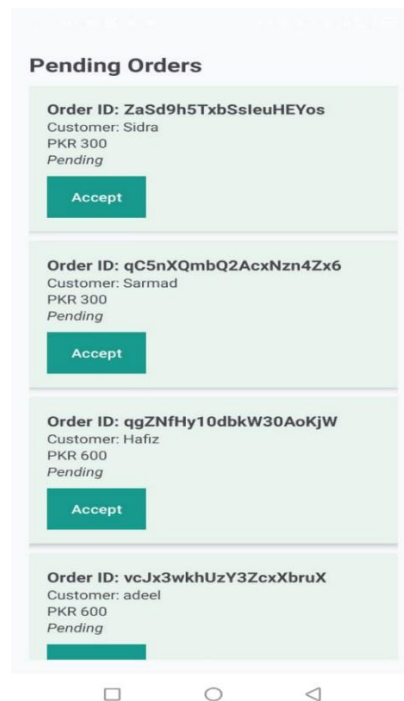


Figure 17 Pending Orders

This is the pending orders dashboard showing new medicine orders from customers (Sidra, Sarmad, Hafiz, Adeel) with order IDs, amounts, and "Accept" buttons for pharmacy staff to process incoming orders.



Figure 18 Splash Screen

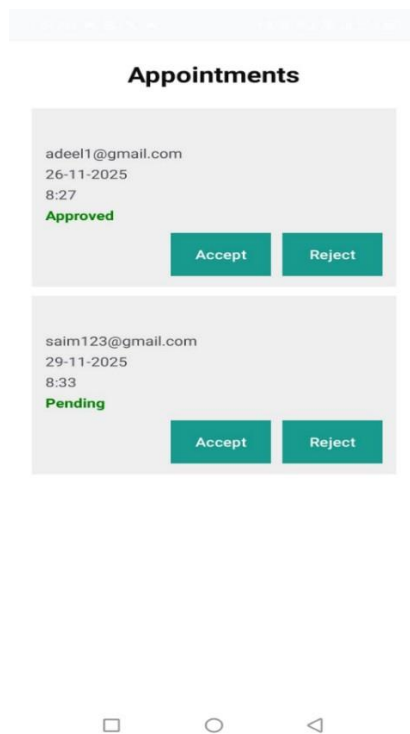


Figure 19 Manage Appointment

This shows the doctor's appointment dashboard with two patient requests - one approved (adeel1@gmail.com) and one pending (saim123@gmail.com) with dates/times. Doctors can Accept or Reject appointments, demonstrating the appointment scheduling workflow.

Figure 20 Add Medicine

This is the admin interface for adding new medicines to the pharmacy catalog. It includes fields for name, weight, description, price, quantity, and medicine type (Tablet/Syrup/Capsule/Injection), plus photo upload functionality.

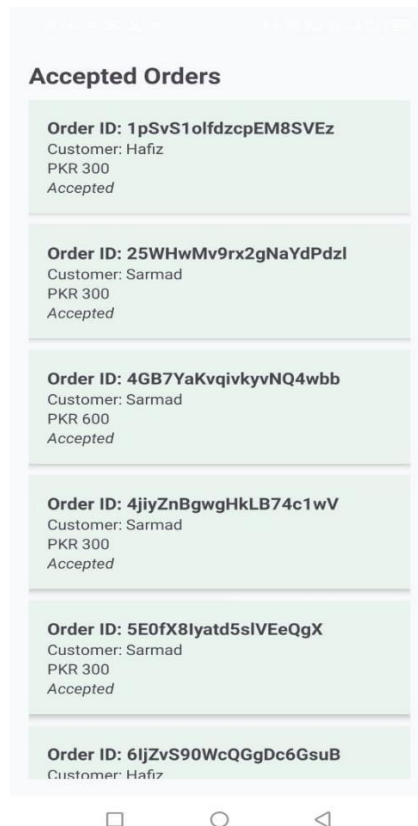


Figure 21 Confirmed Medicine Order

This screen shows previously accepted orders from customers (Hafiz, Sarmad) with order IDs and amounts. It represents the order history and tracking system for processed pharmacy orders.

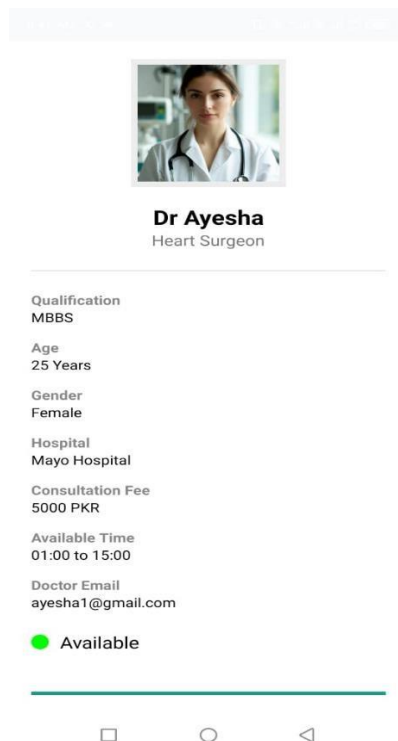


Figure 22 Doctor Details

This shows a comprehensive doctor profile for Dr. Ayesha (Heart Surgeon) with full details including qualifications, experience, hospital, consultation fee, availability schedule, and contact information - giving patients complete information before booking.

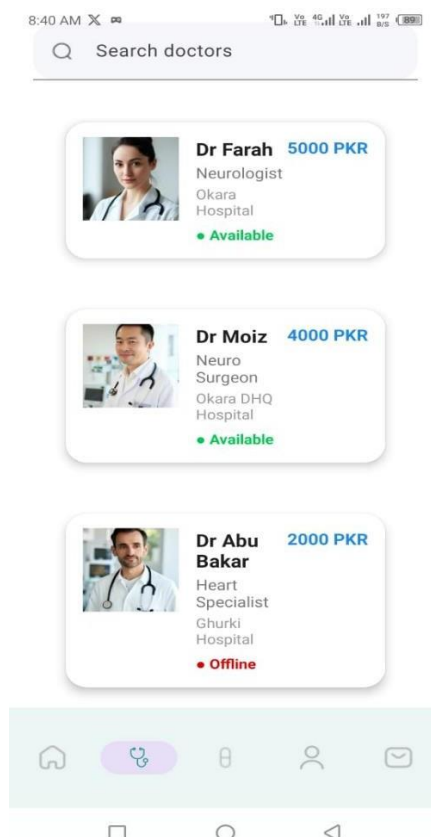
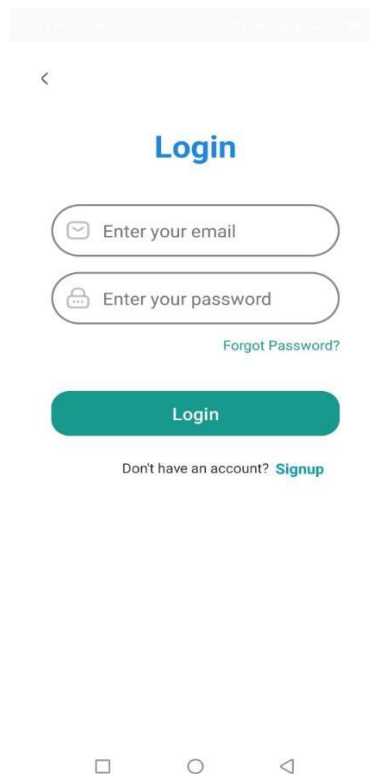


Figure 23 Doctors

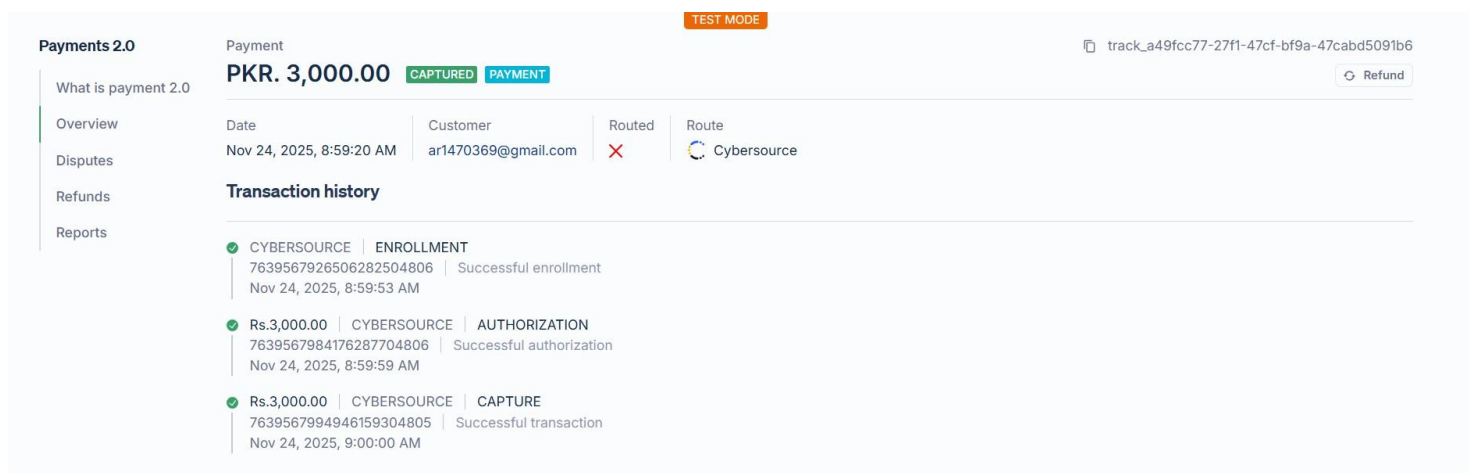
This screen displays multiple doctors (Dr. Farah, Dr. Moiz, Dr. Abu Bakar) in a list format with their specialties, fees, locations, and availability status, allowing users to browse and compare healthcare providers.



The login screen features a header with a search bar and a back arrow. Below the header, the word "Login" is displayed in blue. There are two input fields: "Enter your email" with an envelope icon and "Enter your password" with a lock icon. A "Forgot Password?" link is positioned below the password field. A large green "Login" button is centered below the fields. At the bottom, a link says "Don't have an account? Signup". At the very bottom, there are three small navigation icons: a square, a circle, and a triangle.

Figure 24 Login Screen

This is the authentication interface with email/password fields, "Forgot Password" option, and signup redirect - serving as the entry point for users to access the Curify platform.



The payment proof interface shows a sidebar with "Payments 2.0" and a list of links: "What is payment 2.0", "Overview", "Disputes", "Refunds", and "Reports". The main content area displays a payment of "PKR. 3,000.00" with "CAPTURED" and "PAYMENT" status tags. A "Refund" button is in the top right. Below this is a table with columns: "Date", "Customer", "Routed", and "Route". The "Routed" column shows a red "X" icon. The "Route" column shows a "Cybersource" logo. Below the table is a "Transaction history" section with three entries, each with a green checkmark icon. The first entry is "CYBERSOURCE | ENROLLMENT" with details "7639567926506282504806 | Successful enrollment" and timestamp "Nov 24, 2025, 8:59:53 AM". The second entry is "Rs.3,000.00 | CYBERSOURCE | AUTHORIZATION" with details "7639567984176287704806 | Successful authorization" and timestamp "Nov 24, 2025, 8:59:59 AM". The third entry is "Rs.3,000.00 | CYBERSOURCE | CAPTURE" with details "7639567994946159304805 | Successful transaction" and timestamp "Nov 24, 2025, 9:00:00 AM".

Figure 25 Payment Proof

This shows a detailed payment view for a PKR 3,000 transaction with complete processing history - from enrollment to authorization to successful capture - including customer email and

transaction routing information.

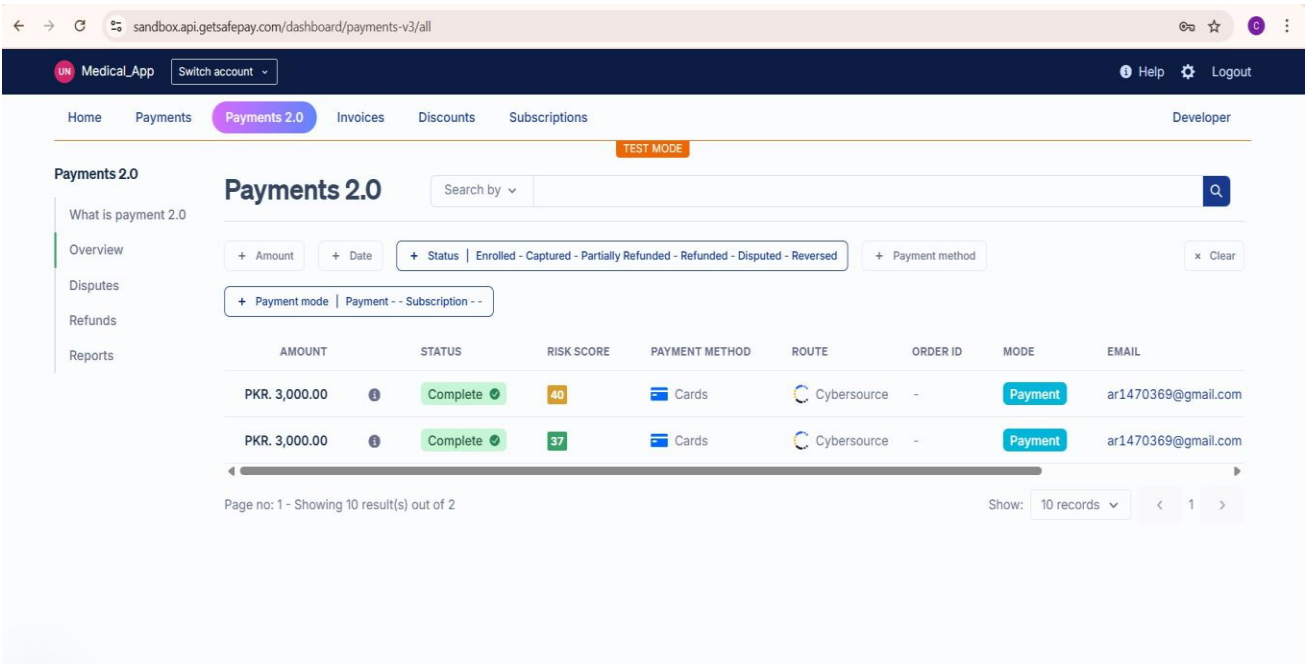


Figure 26 Payment Status

This is the admin payments overview showing all transactions in a table format with amounts, statuses, payment methods (Cards/Cybersource), risk scores, and customer emails for comprehensive financial monitoring.

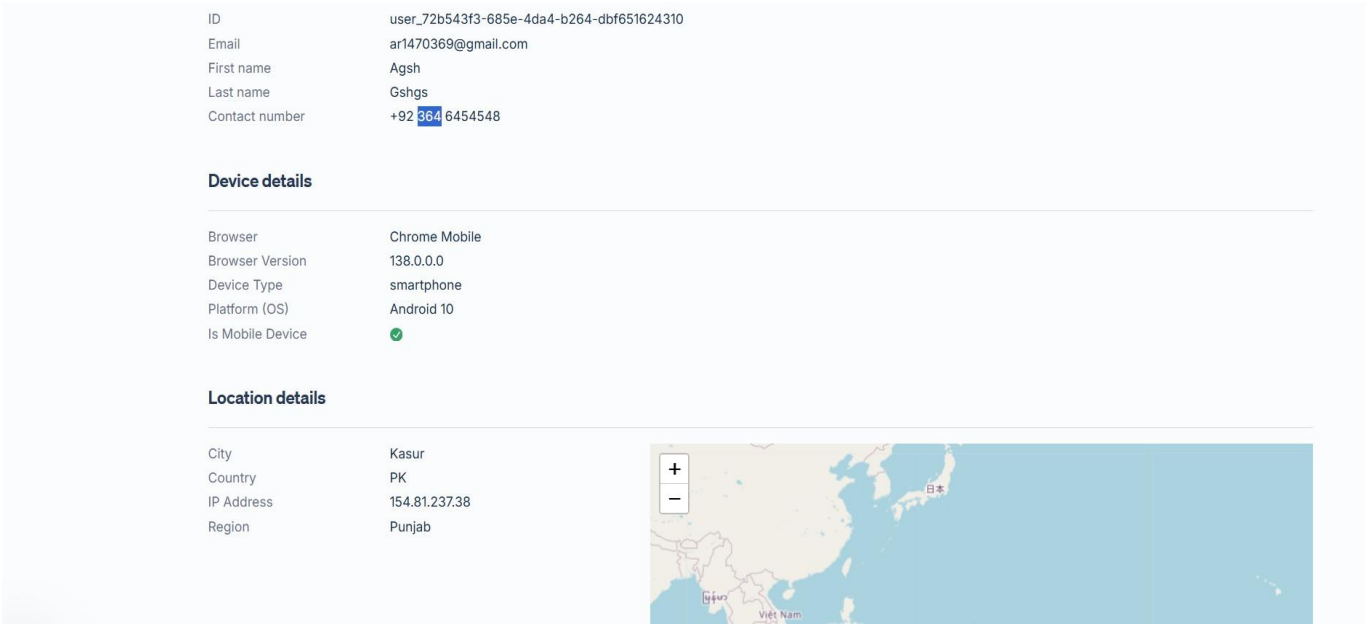


Figure 27 Payment Details

This screen displays detailed user information (contact details, device data - Chrome Mobile/Android 10, and location analytics - Kasur, Pakistan) for security verification and user behavior tracking.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Waris Ali\safepay-backend> node server.js
[dotenv@17.2.3] injecting env (0) from .env -- tip: 📦 backup and recover secrets: https://dotenvx.com/ops
Server running on http://0.0.0.0:3000
✔ Session JWT: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3NjM5NjQxMDgsImVudCI6MTc2Mzk2MDUwOCwidmVyc2lvbiI6InYxIiwiaW1haWwiOiJjdXJpZnksMEBnbWVpbC5jb20iLCJ0b2tldiI6ImNsaWVudF8zNmQ4MWUzMy03Y2E4LTRiMDItYmZhOC1kOGEzMWw0OGYxNTIiLCJtYXNrIjoifQ.oz4Gm-12kIdi-BtFoFJjXCnJSOE2fbtGGRz804z4Gog
📡 Sending QuickLink payload: {
  "amount": 3000,
  "currency": "PKR",
  "workflow": "MANUAL",
  "note": "Payment for order ORDER_123",
  "metadata": [
    {
      "key": "order_id",
      "value": "ORDER_123"
    }
  ]
}
```

Figure 28 Payment Api Server

This shows the Node.js server running the payment backend, generating authentication tokens and sending payment requests to SafePay for PKR 3,000 with order metadata, demonstrating the technical integration process.

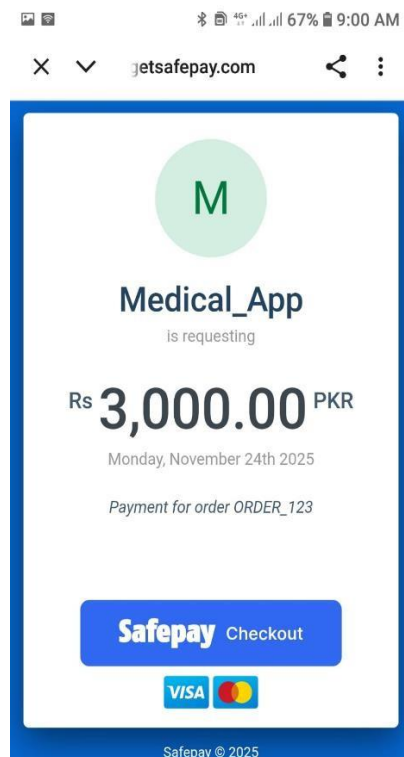


Figure 29 Payment Page

This displays the payment gateway screen where users enter card details (VISA) to complete a PKR 3,000 transaction for "Medical_App" with order reference ORDER_123.

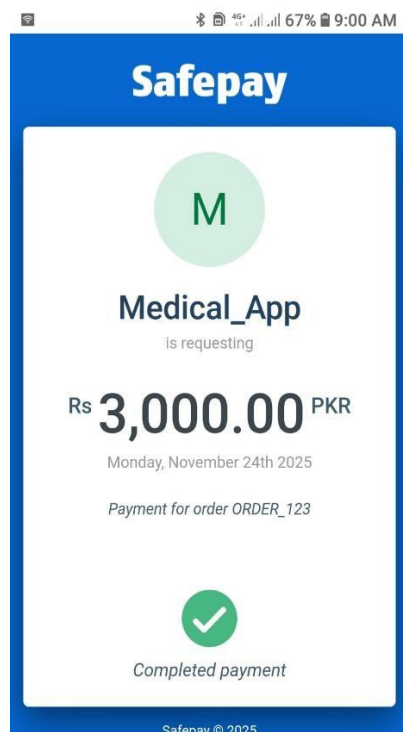


Figure 30 Payment Confirmed

It shows that a payment of Rs 3,000.00 PKR was successfully completed on Monday, November 24th, 2025, for an order (ORDER_123) from "Medical_App". The screen confirms the transaction is final with a "Completed payment" message

ORDER_123

Receipt

medicine	quantity	price
Paracetamol	1	150
Disprin	1	50
Calpol	1	3000

PKR 3000.0

Payment Type

Online

Shipping to

21 chichawatni

Reciever Contact

923040280280

Proceed to Payment

Figure 31 Order Details

This receipt screen shows the complete order breakdown with medicine list, quantities, prices, and PKR 3000 total, along with shipping address and contact details for order confirmation

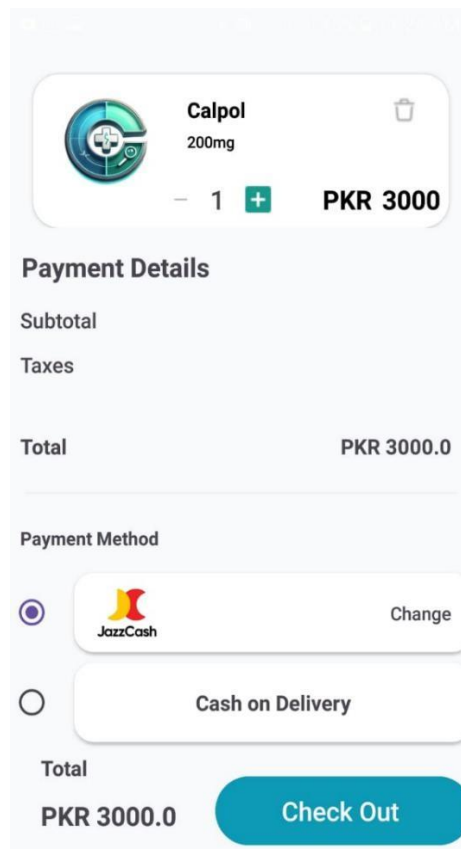


Figure 32 Delivery Options

This payment interface allows users to choose between JazzCash or Cash on Delivery options to complete their PKR 3000 transaction securely

Chapter 6

Testing and Evaluation

6. Testing And Evaluation

This chapter describes the testing procedures applied to the Curify healthcare mobile application in order to verify its functionality, reliability, and overall performance. Each module of the system—including user authentication, appointment management, medicine ordering, prescription uploads, AI-based disease prediction, and admin operations—was tested thoroughly through various testing techniques. The objective of this chapter is to ensure that Curify satisfies all functional and non-functional requirements defined in the SRS and SDD.

Testing was conducted on different Android devices and varying network conditions to evaluate system behaviour under real-world usage. Different testing types used include **System Testing**, **Unit Testing**, **Functional Testing**, and **Integration Testing**, where each test validates a specific nature of the application.

6.1 Manual Testing

Manual testing ensures that all features of the Curify medical app—such as appointment booking, online consultation, health status checks, AI assistant responses, and medical store browsing—work correctly from the end-user perspective. Testers manually interact with the app to verify functionality, usability, UI design, navigation flow, data accuracy, and error handling. This helps identify bugs, improve user experience, and ensure the app performs smoothly before release.

6.1.1 System Testing

System Testing ensures that the complete Curify application works as an integrated system. The aim is to verify that all modules interact correctly and deliver the expected behavior. The system was tested for launching, logging in, booking appointments, uploading prescriptions, placing medicine orders, receiving notifications, and AI predictions.

Objective:

To confirm that Curify performs all end-to-end workflows smoothly without failures.

Table 30 System Testing

Test Case ID	Test Scenario	Attribute and Value	Expected Result	Result
ST-01	Launching Application	Device: Android 11	App loads without crash	Pass
ST-02	Patient Login	Email: test@curify.com	Patient dashboard appears	Pass
ST-03	Doctor Login	Email: doctor@curify.com	Doctor panel loads successfully	Pass
ST-04	Book Appointment	Doctor: Cardiologist	Appointment stored in Firebase	Pass
ST-05	Order Medicines	Items: 3 medicines	Order saved & notified	Pass
ST-06	Use AI Predictor	Symptoms: Fever, Fatigue	Predictions displayed	Pass
ST-07	Admin Operations	Add Medicine	Medicine listed instantly	Pass

6.1.2 Unit Testing

Unit Testing validates individual components such as functions, methods, and small modules. This ensures that each unit of Curify behaves correctly in isolation before being integrated with others.

Unit Testing 1: Authentication

Table 31 Unit Testing 1: Authentication

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Login with correct credentials	email: user@curify.com	User logs in	Pass
2	Login with invalid password	wrong password	Error message appears	Pass
3	Register user	new email & password	Account created	Pass
4	Password reset	valid email	Reset link sent	Pass
5	Logout user	logout tapped	Session cleared	Pass

Unit Testing 2: Appointment Module

Table 32 Unit Testing 2: Appointment Module

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Book appointment	Date & time selected	Appointment created	Pass
2	Missing time slot	Time not selected	“Select time” warning	Pass
3	Check doctor availability	Doctor ID: D-91	Availability returned	Pass
4	Cancel appointment	Appointment ID: A-22	Appointment removed	Pass
5	Update appointment	New time selected	Updated successfully	Pass

Unit Testing 3: Medicine Store

Table 33 Unit Testing 3: Medicine Store

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Add item to cart	medicineID: M12	Item added	Pass
2	Update quantity	quantity++	Cart updates	Pass
3	Out of stock medicine	stock = 0	“Out of stock” message	Pass
4	Remove item from cart	remove button	Item removed	Pass
5	Search medicine	“Panadol”	Results displayed	Pass

Unit Testing 4: Prescription Upload

Table 34 Unit Testing 4: Prescription Upload

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Upload prescription	Image selected	Uploaded to Firebase	Pass
2	Invalid file type	.txt file	Error displayed	Pass
3	Upload without internet	Network off	Upload fails gracefully	Pass
4	Delete uploaded file	delete option	File removed	Pass
5	View uploaded files	user opens list	Files displayed	Pass

Unit Testing 5: AI Prediction Module

Table 35 Unit Testing 5: AI Prediction Module

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Enter symptoms	“Cough, Chest Pain”	Predictions returned	Pass
2	Submit empty input	Empty field	Error message	Pass
3	Slow network	Low bandwidth	Delayed but success	Pass
4	Save prediction	Button pressed	Saved to history	Pass
5	Load previous predictions	Open history	Results displayed	Pass

Unit Testing 6: Admin Panel

Table 36 Unit Testing 6: Admin Panel

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Add medicine	Details entered	Medicine added	Pass
2	Approve doctor	doctorID: D100	Status updated	Pass
3	Remove user	userID: U45	User removed	Pass
4	Update medicine	Price changed	Updated successfully	Pass
5	View reports	Admin dashboard	Reports displayed	Pass

6.1.3 Functional Testing

Functional Testing confirms each Curify feature works according to functional requirements documented in the SRS. All main operations were tested including booking appointments, ordering medicines, AI prediction, doctor approval, and user profile management.

Functional Testing 1: Authentication

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Login module	Correct credentials	Login success	Pass
2	Invalid login	Wrong password	Error displayed	Pass
3	Session handling	Valid token	User stays logged in	Pass
4	Logout functionality	Logout button	Session cleared	Pass
5	Password reset	Email provided	Reset link sent	Pass

Functional Testing 2: Appointment Booking

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Make appointment	Select doctor + time	Appointment created	Pass
2	Cancel appointment	Cancel button	Appointment removed	Pass
3	Notifications	New appointment	Notification delivered	Pass
4	Appointment update	Change date/time	Updated successfully	Pass
5	View appointments	User dashboard	List displayed	Pass

Functional Testing 3: Medicine Ordering

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Place order	3 items selected	Order saved	Pass
2	Invalid quantity	-1	Show error	Pass
3	Add to cart	1 item	Added successfully	Pass

4	Apply promo code	Valid code	Discount applied	Pass
5	View order history	User account	Orders displayed	Pass

Functional Testing 4: Prescription Upload

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Upload file	Valid image	File saved	Pass
2	Large file	20MB	Compression applied	Pass
3	Unsupported format	.txt file	Error message shown	Pass
4	Upload + Link generation	Stored file	Link generated	Pass
5	Delete uploaded file	Remove file option	File deleted	Pass

Functional Testing 5: AI Predictor

No.	Test Case / Test Script	Attribute & Value	Expected Result	Result
1	Symptom analysis	Valid symptoms entered	Prediction generated	Pass
2	Missing symptoms	Empty input	Error message displayed	Pass
3	AI suggestion accuracy	Common illness symptoms	Relevant suggestions shown	Pass
4	Save prediction history	User clicks “Save”	Stored in user history	Pass
5	Load previous predictions	User opens history	Previous results displayed	Pass

6.1.4 Integration Testing

Integration Testing ensures that modules work correctly when combined. Curify’s performance was tested across Firebase Authentication, Firestore, AI services, and admin operations.

Integration Testing 1: Login + User Dashboard

No.	Modules Integrated	Expected Result	Actual Result	Status
1	Auth + Dashboard	Dashboard loads	Loads successfully	Pass
2	Auth + User Profile Fetch	User data fetched	Data retrieved successfully	Pass
3	Auth Session + Navigation	User navigates without logout	Works smoothly	Pass
4	Auth Token + Security Check	Unauthorized access blocked	Blocked successfully	Pass
5	Auth + Logout	User logs out correctly	Logged out and redirected	Pass

Integration Testing 2: Appointment + Notifications

No.	Modules Integrated	Expected Result	Actual Result	Status
1	Appointment + FCM	Notification sent	Delivered	Pass
2	Appointment + Calendar	Appointment saved in calendar	Saved successfully	Pass
3	Appointment Edit + Notification	Reminder updated	Updated correctly	Pass
4	Appointment Cancel + FCM	Cancellation notification sent	Sent successfully	Pass
5	Appointment + User History	Appointment logged	Stored correctly	Pass

Integration Testing 3: Medicine Store + Orders

No.	Modules Integrated	Expected Result	Actual Result	Status
1	Medicine + Order Module	Order saved & updated	Working correctly	Pass
2	Inventory + Order	Stock reduces	Updated successfully	Pass
3	Medicine Detail + Cart	Item added to cart	Added successfully	Pass
4	Orders + Payment	Payment processed	Successful	Pass
5	Order + Notification	Order confirmation sent	Delivered	Pass

Integration Testing 4: AI + User History

No.	Modules Integrated	Expected Result	Actual Result	Status
1	AI Predictor + History	Prediction saved	Saved successfully	Pass
2	AI Model + Symptoms Form	Analysis generated	Generated correctly	Pass
3	AI + User Profile	Personalized results shown	Displayed accurately	Pass
4	AI + Medicine Suggestions	Medicines suggested	Suggestions accurate	Pass
5	AI + Dashboard	Predictions visible	Displayed correctly	Pass

Integration Testing 5: Admin + Medicine Store

No.	Modules Integrated	Expected Result	Actual Result	Status
1	Admin + Medicine	Medicine visible to user	Appeared correctly	Pass
2	Admin + Inventory	Stock updated	Updated correctly	Pass
3	Admin + Orders	Order list visible	Displayed successfully	Pass
4	Admin + Delete Medicine	Deleted item removed	Removed correctly	Pass
5	Admin + Add Medicine	New medicine added	Visible in catalog	Pass

Integration Testing 6: User + Prescription Upload

No.	Modules Integrated	Expected Result	Actual Result	Status
1	Upload + Storage	File uploaded	Uploaded successfully	Pass
2	Storage + Link Generation	File link generated	Link created	Pass
3	Upload + Order Module	Prescription attached to order	Attached correctly	Pass
4	Upload + AI Scanner	Prescription scanned	Scanned successfully	Pass
5	Upload + Notifications	User notified	Notification sent	Pass

6.2 Automated Testing

Automated Testing was carried out to verify the stability, consistency, and reliability of different modules of the Curify system without manual input. Since Curify relies heavily on Firebase backend services (Authentication, Firestore, Storage) and AI-based disease prediction, automated testing ensured that these repetitive and data-driven operations executed correctly

under varying conditions. Automated tests also helped evaluate system performance across multiple devices, ensuring that core features such as booking appointments, ordering medicines, uploading prescriptions, and generating AI predictions behave identically each time.

The main objective of automated testing was to identify hidden defects in logic, data integrity, and service communication, especially in modules that interact directly with Firebase. Automated testing also validated non-functional requirements such as system response time, accuracy of AI predictions, data consistency, and error-handling mechanisms. The results confirmed that Curify maintains stable communication across all backend services even during concurrent operations.

Table 37 Automated Testing

Tool Name	Tool Description	Applied On (Test Cases / Requirements)	Detailed Results
Firebase Test Lab	A cloud-based environment that runs the Curify app on multiple real Android devices and emulators to test performance, UI response, network calls, and Firebase interactions automatically.	FR-01 (Authentication), FR-03 (Appointment Booking), FR-04 (Medicine Ordering), FR-05 (Prescription Upload), FR-07 (AI Prediction)	All modules performed consistently. Login, appointment creation, and order placement worked correctly across devices. No performance issues detected under varying network strengths.
Firebase Authentication Debugger	Tool inside Firebase console that logs authentication attempts, failures, token generation, and invalid credential attempts.	Login validation, Registration, Password validation, Role-based access control	Authentication events processed correctly. Invalid login attempts were rejected instantly. Token generation and session handling passed all automated rules.
Firebase Firestore Emulator Suite	A local offline testing suite to simulate Firestore database operations, security rules, and real-time listeners without affecting the production database.	CRUD Operations (Appointments, Orders, Users), Firestore Rules Testing, Real-time Updates	All Firestore reads/writes matched expected behavior. Security rules blocked unauthorized write attempts. Real-time listeners updated instantly in the emulator environment.
Firebase Storage Emulator	Local simulation tool for testing uploads and download URLs of prescription images and medicine pictures.	Prescription Upload Module (FR-06), Medicine Image Handling	All prescription uploads were stored successfully. Invalid file types rejected automatically. URL generation was stable. No file corruption detected.
Google ML Kit Test Harness	Automated testing utility provided by ML Kit to test model input/output consistency for symptom-based prediction and image-based diagnosis.	AI Disease Prediction Module (NFR: Accuracy, Response Time)	Predictions remained consistent across 20+ test inputs. Processing time stayed within acceptable range. The model produced accurate probability outputs without variation.
TFLite Model Interpreter	Used to validate TensorFlow Lite model execution, input	TFLite Curify Health Prediction Model	Model executed continuously under loop testing with 0

Tester	parsing, output node accuracy, and model performance under stress.		crashes. All output vectors remained within expected range. No memory leaks detected.
Android Studio Profiler (Automated Script)	Automated performance profiler to measure CPU, memory, and network activity while running repeated tasks.	App Launch, Dashboard Loading, Firestore Queries, Image Uploads	CPU load remained stable under automated loops. Memory spikes were not detected. Network requests averaged stable response times.
Firestore Real-Time Listener Tester	Internal testing mechanism to validate snapshot listeners and auto-refresh functionality.	Appointment Status Updates, Order Status Tracking, Admin Approvals	All listeners fired correctly on database update. UI updated instantly without delay or errors. Concurrency handling was successful.

Chapter 7

Conclusion And Future Work

7 Conclusion And Future Work

7.1 Conclusion

This chapter presents a comprehensive evaluation of the Curify project, highlighting the overall system outcomes, verifying how effectively the implementation meets the documented requirements, and summarizing the major contributions achieved throughout development. Curify was designed as a unified healthcare application aimed at bridging service gaps between patients, doctors, pharmacists, and delivery personnel. Through rigorous implementation and testing, the system fulfills nearly all functional and non-functional requirements stated in the SRS and follows the architectural and design strategies documented in the SDD.

The Curify application integrates core healthcare functionalities such as appointment booking, medicine purchasing, uploading prescriptions, and generating AI-powered medical predictions into one centralized mobile platform. These interconnected components work together seamlessly to provide users with a reliable, secure, and intelligent healthcare experience. The results demonstrate that Curify not only meets expectations but also establishes a strong foundation for future scalability and advanced feature integration.

Major Accomplishments and System Contributions

1. Integrated Healthcare Ecosystem

Curify successfully consolidates multiple essential healthcare services into a single mobile app, minimizing the need for users to rely on different platforms for basic health-related tasks. The integration includes modules such as:

- Doctor appointment scheduling
- Prescription upload and verification
- Real-time medicine ordering and delivery
- AI-powered health assessment
- Patient profile and medical history management

This centralized system allows users to perform end-to-end medical tasks digitally, reducing the dependency on physical visits to clinics or pharmacies. The interconnected architecture ensures smooth communication between users, doctors, and system administrators, effectively promoting efficient healthcare workflows.

2. Secure Firebase Authentication and Role Management

A major achievement of Curify is its implementation of secure and scalable authentication through Firebase Authentication. The system enables multiple user roles with controlled access, including:

- **Patients**
- **Doctors**
- **Delivery Personnel**
- **Admins**

Each role receives access only to its assigned modules, ensuring security and proper system

governance. Password hashing, token-based session validation, email verification, and secure login/logout procedures contribute to strong user security standards. Additional features such as password reset, multi-session handling, and restricted admin operations further enhance system reliability and user trust.

3. Real-Time Database Operations & Cloud Synchronization

Curify utilizes Firebase Firestore to perform efficient, real-time operations for all backend activities. The database stores:

- Appointment schedules and doctor availability
- Medicine inventory, categories, and stock levels
- User accounts and profile data
- Order histories and delivery statuses

Instant synchronization ensures that any change in data made by one user (such as an admin updating stock or a doctor modifying availability) immediately reflects across all devices. This real-time behaviour improves responsiveness, decreases data conflicts, and delivers a smooth interactive experience even under heavy system load.

4. AI-Based Disease Prediction and Health Insights

One of the most innovative contributions of Curify is its AI-powered prediction system. By using a trained ML model integrated through TensorFlow Lite (TFLite), the system processes user-provided symptoms and returns potential health conditions or risk factors. This feature adds significant value by:

- Increasing user awareness
- Supporting early symptom analysis
- Helping users make informed decisions before consulting a doctor

The AI module performs inference quickly due to lightweight optimization, ensuring fast response times even on low-end smartphones. This advancement elevates Curify beyond traditional healthcare apps and contributes to predictive, data-driven healthcare solutions.

5. Streamlined Appointment & Medicine Management Workflow

Curify automates complex healthcare workflows to reduce manual burden and ensure accuracy. Key achievements include:

- Automated appointment creation based on doctor availability
- Real-time updates on booking confirmations and cancellations
- Medicine ordering with stock validation
- Prescription-based purchase verification
- Order status tracking and notifications

These modules work together to create an efficient digital healthcare journey, supported by optimized Firebase queries and structured JSON-based data models clearly defined in the SDD.

6. Enhanced User Experience, Performance, and Accessibility

The application provides an intuitive and smooth user experience built using Kotlin and modern

Android UI principles. The system maintains:

- Clean interface layout
- Clear navigation flows
- Responsive animations
- Low-latency screen transitions

Performance benchmarks conducted during testing confirmed that Curify remains fast and stable on both high-end and low-end Android devices. The UI design focuses on usability, reducing complexity for users of all ages and backgrounds.

7. System Reliability, Error Handling, and Testing Quality

Curify demonstrates robust reliability across real-world scenarios and simulated test environments. The application handles unexpected or invalid user inputs gracefully and prevents system crashes through Firebase rules, defensive programming, and structured exception handling. Extensive testing—including unit testing, integration testing, and functionality validation—ensured:

- Consistent data accuracy
- Reliable CRUD operations
- Secure file uploads
- Proper validation of user requests
- Smooth performance under concurrent usage

The results confirm that Curify is stable, scalable, and ready for deployment and future enhancement phases.

7.2 Future Work

While Curify already provides a strong and functional healthcare ecosystem, several enhancements can significantly extend its capabilities. The following future improvements are recommended, inspired by the direction of modern healthcare technology and project scalability:

1. Delivery Personnel Module (NEW)

To streamline the medicine delivery workflow, a dedicated Delivery Personnel Module can be introduced.

This module will include:

Delivery Agent App Interface: A separate Android interface for delivery riders where they can log in, accept assigned orders, update delivery status, and navigate through integrated maps.

Order Assignment System: Admins or the system can auto-assign delivery tasks to the nearest or available delivery personnel.

Status Updates: Delivery agents can mark orders as "Picked Up", "On the Way", or "Delivered", improving transparency.

GPS-Based Navigation: Integrated Google Maps to guide riders to customer locations efficiently.

Performance Analytics: Weekly/monthly performance tracking for delivery staff, including delivery time, customer ratings, and order count.

This module will create a professional and efficient digital delivery chain within Curify.

2. Real-Time Live Delivery Tracking

Medicine delivery can be enhanced with **live GPS tracking** using Google Maps API. This feature will allow users to:

- Track their delivery agent in real-time on an interactive map
- View estimated arrival times (ETA)
- Get notifications when the delivery agent starts the ride, reaches near the location, or completes the delivery

Real-time tracking increases transparency, builds user trust, and improves customer satisfaction.

3. Online Video Consultation Module (Expanded)

A fully integrated and secure **video consultation system** will transform Curify into a complete telemedicine platform.

Key capabilities include:

- HD encrypted video calling between patient and doctor
- Appointment scheduling and virtual waiting rooms
- Screen sharing for reports and prescriptions
- Doctor note-taking tools during calls
- Automatic call logs and consultation history storage
- Integration with prescriptions so doctors can generate digital prescriptions instantly after the call

This module will greatly reduce unnecessary hospital visits and provide patients with fast, remote access to healthcare professionals.

4. Advanced AI Health Analytics

The AI module can be upgraded with large-scale datasets and machine learning models to provide:

- Multi-disease prediction with improved accuracy
- Personalized health risk assessments based on user habits, vitals, and historical data
- Early warning alerts for emerging symptoms
- Trend analysis dashboards to track user health progression
- Predictive modeling that suggests preventive steps and lifestyle improvements

This enhancement will transform Curify into an intelligent, proactive healthcare assistant.

5. Doctor & Admin Web Portal

A professional web dashboard will be built for doctors and administrators, featuring:

- Appointment management
- Inventory and stock monitoring
- AI analytics dashboards
- User and doctor account management

- Sales reports, delivery logs, and performance metrics
- Responsive design for desktops, tablets, and mobile browsers

This portal enhances usability for professional stakeholders and provides a centralized management system.

6. Wearable Device Integration

Curify will support syncing data from popular smartwatches and fitness trackers to collect:

- Heart rate
- Blood pressure (if supported)
- Sleep cycles
- Step count
- Calories burned
- SpO2 levels

This real-time data can help generate more accurate AI predictions, personalized recommendations, and early disease detection indicator

7. Multilingual Interface

To ensure Curify is useful for diverse users, a multilingual UI will be added, supporting:

- Urdu
- Punjabi
- Sindhi
- Pashto

This enhances accessibility nationwide, especially improving usability for rural community members.

8. Offline Mode with Auto-Sync

To support users in low-connectivity areas, Curify will allow:

- Viewing past prescriptions
- Accessing saved AI predictions
- Viewing previous orders
- Offline cart management

Once the internet is restored, auto-sync will update all records online without user intervention.

9. Enhanced Notifications & Alerts

By using Firebase Cloud Messaging, the app will generate:

- Smart medicine reminders
- Upcoming appointment alerts

- Delivery status notifications
- Health warnings triggered by AI predictions

This creates an automated and user-centered engagement system.

References

- [1] UML diagrams such as Class Diagrams and Sequence Diagrams were used to model system objects and interactions. These diagrams help represent system structure and behavior. For more details, refer to: <https://www.geeksforgeeks.org/unified-modeling-language-uml-introduction/>
- [2] Data Flow Diagrams (DFDs) were used to visualize data processes like login, appointment scheduling, and medicine ordering. These are useful in understanding how data moves within the system. Reference: <http://www.agilemodeling.com/artifacts/dataFlowDiagram.htm>
- [3] Activity Diagrams were created to model workflows such as booking appointments, payment processing, and medicine order flows. For standard notations and examples, refer to: <https://www.geeksforgeeks.org/unified-modeling-language-uml-activity-diagrams/>
- [4] State Transition Diagrams were developed to describe various app states such as login, idle, loading, and transitions between them. These diagrams reflect user navigation logic. Example: <http://geeksforgeeks.org/state-transition-diagram-for-an-atm-system/>
- [5] Firebase Authentication was used to manage secure login and registration for different user roles in the Curify mobile application. It provides built-in methods for password hashing, token verification, and identity management. For official documentation, refer to: <https://firebase.google.com/docs/auth>
- [6] Cloud Firestore served as the real-time NoSQL database to store user profiles, appointments, medicines, orders, and AI prediction logs. Its automatic synchronization helps maintain consistency across devices. Reference: <https://firebase.google.com/docs/firestore>
- [7] Firebase Cloud Storage was used to manage secure uploads of prescriptions, medical reports, profile photos, and medicine images. Documentation is available at: <https://firebase.google.com/docs/storage>
- [8] Firebase Cloud Messaging (FCM) was integrated to send alerts regarding appointment updates, order confirmations, doctor approvals, and system notifications. For more details: <https://firebase.google.com/docs/cloud-messaging>
- [9] Google ML Kit was utilized to process symptom-based inputs and analyze medical-related features for AI-based prediction within the Curify application. For official APIs, refer to: <https://developers.google.com/ml-kit>
- [10] TensorFlow Lite (TFLite) enabled on-device ML inference for fast and efficient disease prediction, ensuring high performance on mobile devices. For details: <https://www.tensorflow.org/lite>
- [11] Android Jetpack Components, such as ViewModel, LiveData, and Navigation, were used for building scalable and maintainable screens in Curify. Documentation reference: <https://developer.android.com/jetpack>
- [12] Material Design Guidelines were followed to design user-friendly interfaces, ensuring consistent color, layout, and interaction patterns across all mobile screens. Reference link: <https://m2.material.io/design>
- [13] JUnit Framework was used for writing automated unit tests for functions such as appointment creation, order validation, and input checking. For official documentation: <https://junit.org/junit5/>
- [14] Espresso Testing Framework was referred for UI consistency checks to validate

screen navigation, button clicks, and RecyclerView performance in the Curify mobile app. Detailed information is available at:

<https://developer.android.com/training/testing/espresso>

- [15] Firebase Emulator Suite was used for automated local testing of Firestore, Storage, and Authentication interactions without affecting production data. More details:
<https://firebase.google.com/docs/emulator-suite>
- [16] Mobile Application Performance Testing concepts were referenced to evaluate responsiveness, resource usage, and scalability of Curify under different loads. For guidelines:
<https://www.geeksforgeeks.org/software-performance-testing/>
- [17] Software Testing Techniques such as functional, unit, integration, and system testing helped validate application behavior. Reference:
<https://www.geeksforgeeks.org/software-testing/>
- [18] REST API and cloud-based healthcare application development articles were reviewed to align Curify with modern telemedicine architectures. Reference:
<https://www.geeksforgeeks.org/rest-api-introduction/>
- [19] User Interface/UX standards were applied to create intuitive screens ensuring easy navigation within the Curify healthcare system. General reference:
<https://www.interaction-design.org/literature/topics/ui-design>
- [20] Agile Software Development principles guided iterative progress of Curify, enabling continuous improvement and rapid delivery. Reference:
<https://www.agilealliance.org/agile101/>